

8장 선과 면의 표현



제 8 장

선과 면의 표현

• 개관 •

컴퓨터 그래픽스에서 3차원 객체를 표현하는 방법은 객체의 경계면을 표현하는 방법과 공간 영역을 객체 영역과 배경 영역으로 분할하는 공간분할 방법이 대표적이며, 그 중 경계표현 방식이 가장 기본적인 표현방법이라 할 수 있다.

경계를 표현하기 위해서는 다양한 선과 면을 표현할 수 있어야 한다. 선과 면은 선분이나 평면과 같이 다항식에 의해 간단하게 표현할 수 있는 경우도 있지만, 임의의 형태를 갖는 곡선이나 곡면은 스플라인을 이용하여 근사화하는 것이 일반적이다.

이 장에서는 이러한 객체 모델링의 기본적 개념을 살펴보고, 다양한 형태의 경계면을 표현하기 위해 필요한 곡선 및 곡면 모델링 방법에 대해 탐구한다.

학습 목표

1. 객체 모델링을 위한 방법을 구분하여 설명할 수 있다.
2. 2차 곡면 방정식을 이용하여 타원체, 원환체 등의 기본적인 입체를 표현할 수 있다.
3. 매개변수 다항식을 이용한 스플라인 곡선 표현을 설명할 수 있다.
4. 다양한 스플라인 곡선의 정의 방법 및 특성을 설명할 수 있다.
5. OpenGL을 이용하여 스플라인 곡선 및 곡면을 표현하는 프로그램을 작성할 수 있다.

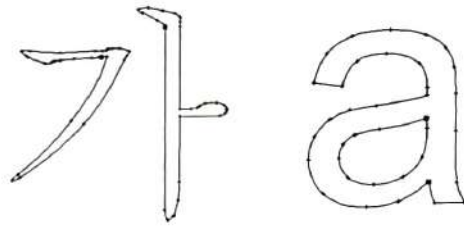
주요 용어

경계표현(b-rep) / 공간분할표현 / 2진 트리 / 구조적 입체기하(CSG) / 스위핑 /
2차 곡면 / 스플라인 / 기저행렬 / 배합함수 / 가하학적 연속성 / 매개변수 연속성 /
에르미트 스플라인 / 베지어 스플라인 / B-스플라인 / NURBS / 아핀 불변성 /
원근투영 불변성

8.1 객체의 모델링

실제 세계에 존재하는 장면에는 건물, 자동차, 가구, 사람, 동물 등 다양한 종류의 객체가 포함되어 있다. 이러한 객체는 다면체나 구, 원기둥 등과 같은 기본적인 기하학적 구조일 수도 있지만, 이러한 간단한 구조로 표현하기 어려운 복잡한 형상일 수도 있다. 컴퓨터 그래픽스에서는 이처럼 복잡한 객체를 컴퓨터를 통해 다룰 수 있는 적절한 모델을 만들고, 이를 바탕으로 하여 근사한 형태로 표현한다.

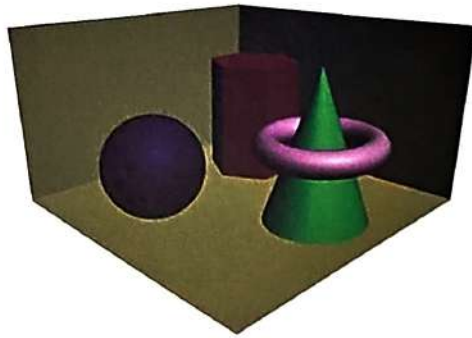
2차원 객체의 경우 평면 위에 표현되는 선분, 다각형, 원, 타원, 포물선 등 정형화된 방정식을 통해 표현하거나, 스플라인 곡선 등을 이용하여 임의 형태의 도형을 근사하게 표현할 수 있다. [그림 8-1]은 베지어(Bézier) 곡선으로 모델링한 글꼴의 예이다.



[그림 8-1] 베지어 곡선으로 표현된 글꼴

3차원 객체인 입체는 물체의 외부와 내부를 나누는 경계면을 표현함으로써 모델링할 수 있다. 이를 경계표현(boundary representation: b-rep이라고도 함)이라고 한다. 3차원 객체는 다면체나 구, 원기둥 등과 같은 기본적인 기하학적 구조일 수도 있고, 좀 더 일반적으로는 표면이 임의 형태의 곡면으로 구성될 수도 있다. 이러한 객체의 표면은 곡면함수를 통해 나타내거나 서로 연결된 다각형이나 스플라인 곡면 등의 집합으로 근사화하여 나타낼 수 있다.

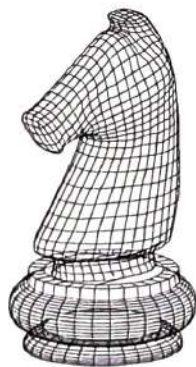
[그림 8-2]는 구, 원뿔, 원환체(torus) 등의 2차 곡면과 육각기둥으로 구성된 장면이다. 이러한 기본적인 다면체나 2차 곡면을 여러 개 조합하면 책상, 건물 등의 외양을 간략하게 표현할 수 있다.



[그림 8-2] 다면체 및 2차 곡면으로 구성된 장면

그러나 표면 구조가 더 복잡한 객체를 표현하기 위해서는 이런 기본적인 곡면만으로는 한계가 있다. 이러한 객체는 다각형이나 곡면 조각을 이용하여 표면을 표현한다. [그림 8-3]의 (가)는 사각형 메시(mesh)를 이용하여 복잡한 형상의 표면을 표현한 예이다. 하나하나의 다각형에 색을 입히면 (나)와 같이 패싯(facets)을 연결한 형태로 보일 것이며, 적절한 음영법(shading)을 활용하면 (다)와 같이 마치 부드러운 곡면인 것처럼 보일 수 있다.

이러한 처리는 다각형 단위로 이루어지기 때문에 만일 다각형의 수를 줄이면 처리시간을 단축시킬 수는 있으나, 표현결과가 조악할 것이다. 반면에, 다각형의 수를 늘려서 더 작은 다각형으로 표현한다면 좀 더 부드러운 표면을 만들 수 있으나, 그만큼 많은



(가) 와이어프레임



(나) 다각형 각면



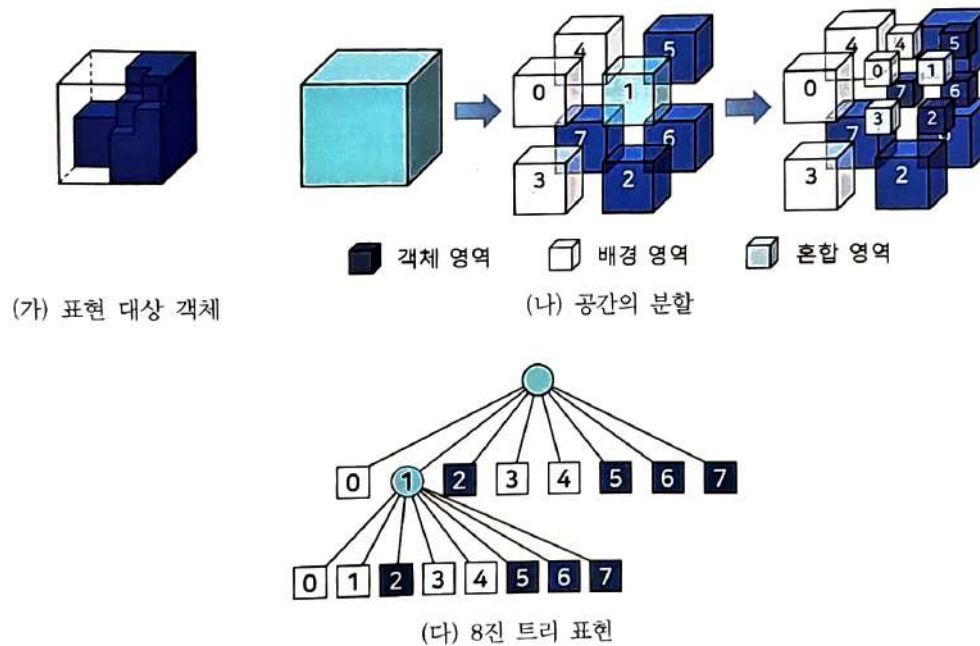
(다) 표면 렌더링 결과

[그림 8-3] 사각형 메시지를 이용한 복잡한 표면의 표현

처리시간을 소비하게 될 것이다.

다각형의 집합으로 객체의 표면을 표현하는 방식은 선형 방정식을 통해 표면을 처리할 수 있게 하므로 컴퓨터 그래픽스에서 매우 유용하게 활용된다. 좀 더 자유로운 표면을 모델링하기 위해서는 NURBS와 같은 스플라인 곡선 및 곡면이 활용될 수 있다.

입체를 표현하는 다른 방법으로는 공간분할표현(space-partitioning representation)이 있다. 이 방법에서는 하나의 입체를 서로 겹치지 않으면서 인접한 입체의 집합으로 표현한다. 대표적인 예인 8진 트리(octree)는 하나의 정육면체를 8개의 정육면체 영역으로 나누어 각 영역이 객체에 완전히 속하는 영역인지, 객체가 속하지 않는 배경 영역인지, 부분적으로 속하는 혼합 영역인지 구분한다. 만일 혼합 영역이라면 이 정육면체 영역을 다시 8개의 정육면체 영역으로 나눈다. 이러한 과정을 반복하여 표현하면 트리 형태의 자료구조가 만들어진다. [그림 8-4]는 8진 트리로 객체를 모델링하는 예이다.

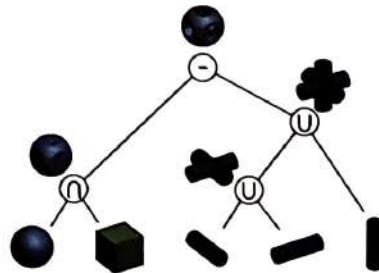


[그림 8-4] 8진 트리를 이용한 객체의 공간분할표현

여러 개의 객체를 불 연산을 통해 조작하여 입체를 모델링하는 방법도 있다. 구조적 입체기하(Constructive Solid Geometry: CSG)는 기본적인 입체들을 불집합 연산을 통해

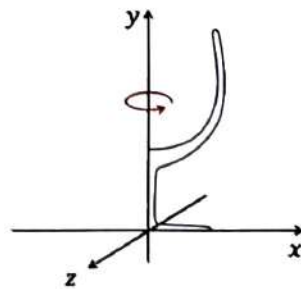
결합한다. 그 결과, 하나의 객체는 트리 형태로 표현되는데, 단말 노드는 단순한 형태의 기본 입체이며, 비단말 노드는 연산자를 저장한다. 연산자는 불집합 연산 또는 이동·회전·크기 조정과 같은 기하변환 연산을 지정한다.

[그림 8-5]는 CSG 트리를 이용하여 모델링한 입체의 예이다. 불집합 연산에는 교환 법칙이 성립하지 않는 것이 있으므로 각 노드의 좌측과 우측 자식 노드는 구분되어야 한다.



[그림 8-5] CSG 트리를 이용한 객체 모델링

스위핑(sweeping) 기법을 이용하여 입체를 모델링하는 방법도 있다. 이 방법은 평면 도형을 설계한 후 이를 3차원 공간상에서 회전이나 이동을 하며 객체를 생성한다. [그림 8-6]은 회전 스위핑을 이용하여 와인 글라스를 모델링하는 예이다.



(가) 2차원 평면도형



(나) 회전 스위핑 결과

[그림 8-6] 회전 스위핑을 이용한 객체 모델링

이외에도 객체를 모델링하기 위한 다양한 기법이 활용되고 있는데, 그중 다각형 메

시나 스플라인 곡면 조각을 이어 붙인 경계표현 방식이 가장 기본적인 방식이다. 이 장에서는 경계표현을 위한 곡면표현 방식에 대해 좀 더 자세히 살펴본다.

8.2 2차 곡면

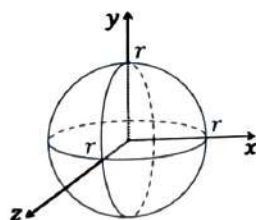
8.2.1. 2차 곡면의 정의

2차 곡면(quadric surfaces)은 2차 방정식으로 묘사되는 곡면이다. 구, 타원면, 원환면 등이 여기에 해당된다. 구는 중심으로부터 거리가 반지름 r 인 좌표점들의 집합으로, 중심이 원점에 있고 반지름이 r 인 구는 식 (8-1)에 의해 정의된다.

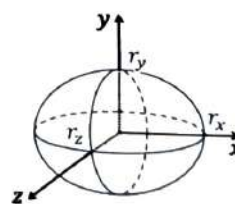
$$x^2 + y^2 + z^2 = r^2 \quad (8-1)$$

타원면은 3차원 데카르트 좌표에서 서로 수직인 각 축 방향의 반지름이 각각 다른 값을 갖는 2차 곡면이다. 식 (8-2)는 중심이 원점에 있고, 각 축 방향의 반지름이 r_x , r_y , r_z 인 타원면을 정의하는 수식이다.

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1 \quad (8-2)$$



(가) 구



(나) 타원면

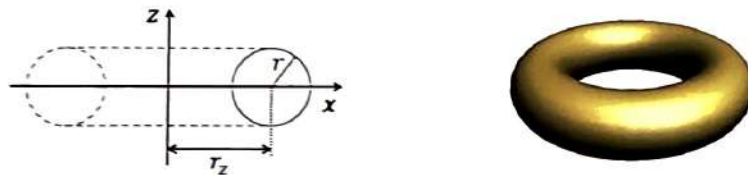
[그림 8-7] 구와 타원면

원환면은 도넛이나 둥근 고리 모양을 의미하는데, [그림 8-8]과 같이 원이나 타원을 외부에 있는 중심축 둘레로 회전하여 만들어지는 2차 곡면이다. 그림에서와 같이 z - x 평면상에 존재하는 반지름이 r 인 원의 방정식은 식 (8-3)과 같다.

$$(x - r_z)^2 + z^2 = r^2 \quad (8-3)$$

이 원을 z 축을 중심으로 회전시킬 경우, 식 (8-3)의 x 대신 회전된 원상의 점 (x, y, z) 와 z 축 사이의 거리인 $\sqrt{x^2 + y^2}$ 을 넣으면 된다. 이때, 원환면의 방정식은 식 (8-4)와 같다.

$$(\sqrt{x^2 + y^2} - r_z)^2 + z^2 = r^2 \quad (8-4)$$



[그림 8-8] 원환면

8.2.2. GLUT 2차 곡면함수

GLUT에서는 구, 원뿔, 원환면 등의 2차 곡면을 손쉽게 생성할 수 있도록 함수를 제공하고 있다. 이 함수를 이용하면 와이어프레임이나 색으로 채워진 표면으로 2차 곡면을 그릴 수 있다.

〈표 8-1〉은 2차 곡면을 생성하는 GLUT 함수이다. `glutWire...`() 함수는 와이어프레임으로, `glutSolid...`() 함수는 지정된 색으로 채워진 표면으로 2차 곡면을 그린다. 각각의 2차 곡면은 원점을 기준으로 정의된다. 또한 원환면과 원뿔은 z 축이 중심축이며, 원뿔의 밑면은 x - y 평면에 위치한다. 그러므로 임의의 위치 및 방향으로 이 2차 곡면을 배치하려면 적절한 기하변환이 필요하다.

[소스코드 8-1]의 `QuadricSurfs.cpp`는 〈표 8-1〉의 2차 곡면을 화면에 디스플레이하는 프로그램이다. [그림 8-9]는 이 프로그램에 의해 만들어진 장면이다.

〈표 8-1〉 GLUT 2차 곡면 함수

종류	GLUT 함수
구	<pre>void glutWireSphere(GLdouble radius, GLint slices, GLint stacks); void glutSolidSphere(GLdouble radius, GLint slices, GLint stacks);</pre> - radius: 구의 반경 - slices: z축에 평행하게 나뉜 구획의 수 - stacks: z축에 수직으로 나뉜 구획의 수
원환면	<pre>void glutWireTorus(GLdouble innerRadius, GLdouble outerRadius, GLint nSides, GLint nRings); void glutSolidTorus(GLdouble innerRadius, GLdouble outerRadius, GLint nSides, GLint nRings);</pre> - innerRadius: 원환면의 안쪽 반지름 - outerRadius: 원환면의 바깥쪽 반지름 - nSides: 각각의 방사선 방향 구획의 면 수 - nRings: 방사선 방향의 구획 수
원뿔	<pre>void glutWireCone(GLdouble base, GLdouble height, GLint slices, GLint stacks); void glutSolidCone(GLdouble base, GLdouble height, GLint slices, GLint stacks);</pre> - base: 원뿔 밑면의 반지름 - height: 원뿔의 높이

162행에서 GLUT_DEPTH 상수로 깊이 버퍼를 사용하도록 하고, 154행에서 깊이 버퍼 가시성 결정을 활성화한 것은 다른 표면 조각에 가려진 면이 보이지 않도록 하기 위한 것으로 제9장에서 가시면 결정과 관련하여 좀 더 자세히 다룰 것이다.

103~110행에서는 정의한 뷰잉 및 투영 변환행렬을 정의하였다. 113행에서는 장면 에 구를 배치하기 위한 이동변환 행렬(modelMat)을 구성하였다. 114~115행에서는 이와 같이 정의한 이동·뷰잉·투영변환이 차례로 적용될 수 있는 복합변환행렬 tMat을 구성하였으며, 116행에서는 tMat을 기하변환행렬 유니폼 변수 gTransform에 전달하였다.

118~120행과 121~123행에서는 면의 색이나 와이어프레임의 색을 프래그먼트 셰이더의 유니폼 변수 glColor를 통해 지정한 후 glutSolidSphere 및 glWireSphere 함수로 구를 그리게 하였다. 125~147행에서는 동일한 방식으로 원뿔과 원환면을 그린다.

[소스코드 8-1] QuadricSurfs.c

```

1  #include <stdio.h>
2
3  #include <gl/glew.h>
4  #include <gl/freeglut.h>
5  #include "gTransform.h"
6
7  typedef struct { // 색을 표현하기 위한 구조체
8      float r, g, b, a;
9  } VecColor4f;
10
11  GLuint uGTransLocation; // 기하변환 행렬 유니폼의 위치
12  GLuint uGColorLocation; // 색 속성 유니폼의 위치
13
14  static const char* pVS = // 꼭짓점 셰이더 소스
15  "#version 330\n"
16  "layout (location = 0) in vec3 Position;\n"
17  "uniform mat4 gTransform;\n"
18  "\n"
19  "void main( )\n"
20  "{\n"
21  "    gl_Position = gTransform * vec4(Position, 1.0);\n"
22  "}";\n"
23
24  static const char* pFS = // 프래그먼트 셰이더 소스
25  "#version 330\n"
26  "out vec4 FragColor;\n"
27  "uniform vec4 gColor;\n"
28  "\n"
29  "void main( )\n"
30  "{\n"
31  "    FragColor = gColor;\n"
32  "}";\n"
33

```

```

34 void AddShader(GLuint shaderProg, const char* pShaderSrc,
           GLint ShaderType)
35 { // 셰이더 생성
...   ※ [소스코드 5-1] (다)의 AddShader 함수와 동일함
56 }
57
58 void SetUpShaders( )
59 { // 셰이더 프로그램 객체 생성
...   ※ [소스코드 5-1] (다)의 SetUpShaders 함수의 70~98행과 동일함
89   uGTransLocation = glGetUniformLocation(shaderProg, "gTransform");
90   uGColorLocation = glGetUniformLocation(shaderProg, "gColor");
91   if (uGTransLocation == 0xFFFFFFFF || uGColorLocation == 0xFFFFFFFF) {
92       fprintf(stderr, "Uniform location not found!!\n");
93       exit(1);
94   }
95 }
96
97 void RenderCB( )
98 {
99     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
100    glClearDepth(1.0);
101    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT); // 화면과 깊이버퍼 지움
102
103    TransMat4f viewMat;
104    Vec3f pOrg = { 2.0f, 2.0f, 2.0f };
105    Vec3f pRef = { 0.0f, 0.0f, 0.0f };
106    Vec3f vUp = { 0.0f, 0.0f, 1.0f };
107    viewingTrans(&viewMat, &pOrg, &pRef, &vUp);
108
109    TransMat4f orthoMat;
110    orthoMatPrj(&orthoMat, -2.0f, 2.0f, -2.0f, 2.0f, 1.0f, 5.0f);
111
112    TransMat4f tMat, modelMat;
113    translate_GTM(&modelMat, 1.0f, 1.0f, 0.0f);

```

```

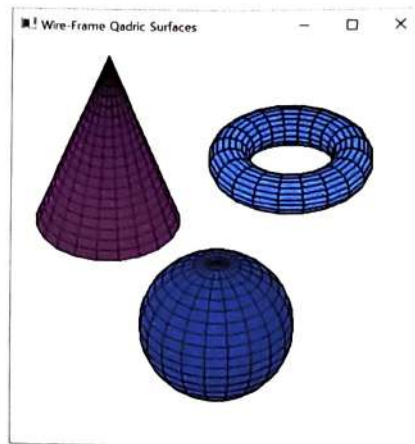
114     tMat = orthoMat;
115     matmul_GTM(matmul_GTM(&tMat, &viewMat), &modelMat);
116     glUniformMatrix4fv(uGTransLocation, 1, GL_TRUE, tMat.mat);
117
118     VecColor4f col = { 0.0f, 0.5f, 1.0f, 1.0f };
119     glUniform4fv(uGColorLocation, 1, &col.r);
120     glutSolidSphere(0.75, 20, 20);
121     col.r = col.g = col.b = 0.0f;
122     glUniform4fv(uGColorLocation, 1, &col.r);
123     glutWireSphere(0.75, 20, 20);
124
125     translate_GTM(&modelMat, 1.0f, -0.5f, 0.5f);
126     tMat = orthoMat;
127     matmul_GTM(matmul_GTM(&tMat, &viewMat), &modelMat);
128     glUniformMatrix4fv(uGTransLocation, 1, GL_TRUE, tMat.mat);
129
130     col.r = 0.7f, col.g = 0.0f, col.b = 1.0f;
131     glUniform4fv(uGColorLocation, 1, &col.r);
132     glutSolidCone(0.7, 2.0, 20, 20);
133     col.r = col.g = col.b = 0.0f;
134     glUniform4fv(uGColorLocation, 1, &col.r);
135     glutWireCone(0.7, 2.0, 20, 20);
136
137     translate_GTM(&modelMat, 0.0f, 1.0f, 1.5f);
138     tMat = orthoMat;
139     matmul_GTM(matmul_GTM(&tMat, &viewMat), &modelMat);
140     glUniformMatrix4fv(uGTransLocation, 1, GL_TRUE, tMat.mat);
141
142     col.r = 0.5f, col.g = 0.7f, col.b = 1.0f;
143     glUniform4fv(uGColorLocation, 1, &col.r);
144     glutSolidTorus(0.2, 0.6, 20, 20);
145     col.r = col.g = col.b = 0.0f;
146     glUniform4fv(uGColorLocation, 1, &col.r);
147     glutWireTorus(0.2, 0.6, 20, 20);

```

```

148
149     glFinish();
150 }
151
152 void Init()
153 {
154     glEnable(GL_DEPTH_TEST);
155     glEnable(GL_POLYGON_OFFSET_FILL);
156     glPolygonOffset(1.0, 1.0);
157 }
158
159 int main(int argc, char** argv)
160 {
161     glutInit(&argc, argv);
162     glutInitDisplayMode(GLUT_RGB | GLUT_DEPTH);
163     glutInitWindowPosition(50, 100);
164     glutInitWindowSize(400, 400);
165     glutCreateWindow("Wire-Frame Quadric Surfaces");
166
167     GLError s = glewInit();
168     if (s != GLEW_OK) {
169         fprintf(stderr, "오류 - %s\n", glewGetErrorString(s));
170         return 1;
171     }
172
173     SetUpShaders();
174     Init();
175
176     glutDisplayFunc(RenderCB);
177     glutMainLoop();
178
179     return 0;
180 }

```



[그림 8-9] GLUT를 이용하여 그린 2차 곡면

8.3 스플라인 곡선 및 곡면

8.3.1. 개요

간단한 다면체나 원기둥, 구, 타원체 등과 같은 정형화된 입체는 비교적 쉽게 정의할 수 있다. 그러나 사람의 얼굴이나 동물, 자동차 등과 같이 부드러운 곡선이나 곡면으로 표현되어야 하는 객체의 경우에는 하나의 공식으로 쉽게 표현할 수 없다. 이러한 곡선이나 곡면은 선분이나 다각형을 이어 붙임으로써 비슷하게 표현할 수 있다. 그러나 이 경우 각진 모서리가 만들어지므로 부드러운 곡선이나 곡면을 만들기 위해서는 조각이 많이 필요하고, 이를 일일이 지정하여 객체를 표현하려면 너무 많은 노력이 필요하다.

스플라인(spline)은 다양한 형태의 부드러운 곡선 및 곡면을 모델링하는 데 유용하게 활용된다. 스플라인은 제도 용어에서 지정된 제어점을 잇는 매끄러운 곡선을 그리기 위한 도구를 의미한다. 컴퓨터 그래픽스에는 정해진 연속성 조건을 만족하는 여러 개의 곡선 조각으로 구성된 곡선을 스플라인 곡선이라고 하며, 각 곡선 조각은 다항식으로 표현된다. 스플라인 곡면은 이러한 스플라인 곡선을 3차원 공간상에서 교차시켜 얻

을 수 있다. 이 장에서는 몇 가지 종류의 대표적인 스플라인에 대해 살펴보고 한다. 우선 스플라인 곡선에 대해 살펴본 후, 이를 곡면으로 확장해 볼 것이다.

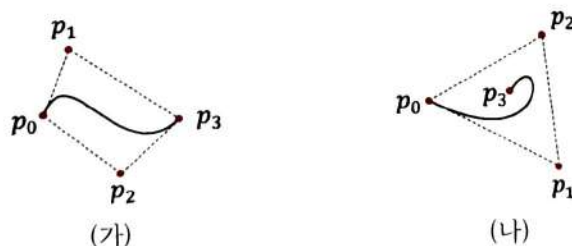
스플라인의 종류는 다양한데, 이들은 공통적으로 몇 개의 제어점(control point)을 이용하여 곡선의 모양을 결정하는 다항식을 만든다. 제어점은 공간상의 좌표 위치인데, 다항식이 제어점들을 모두 연결한다면, 이 곡선은 제어점을 보간(interpolation)한다고 말한다. 만일 제어점들이 곡선 경로상에 존재할 필요 없이 제어점들을 매끄럽게 표현 하도록 곡선이 만들어진다면, 이 곡선은 제어점들을 근사화하는 근사곡선(그림 8-10 참조)이 된다.



[그림 8-10] 곡선 맞춤의 형태(●: 제어점)

제어점들의 집합으로부터 이를 통해 정의되는 곡선의 공간 영역 경계를 나타내는 볼록 껍질(convex hull)을 구할 수 있다. 볼록 껍질은 제어점들을 둘러싸는 최소 면적의 볼록다각형 경계를 나타낸다. 볼록 껍질의 모양은 제어점들이 모두 포함되도록 크게 늘린 고무밴드를 풀어 놓았을 때 제어점들에 걸쳐 있는 고무밴드를 상상해 보면 알 수 있다.

[그림 8-11]은 p_0, p_1, p_2, p_3 에 의해 정의된 스플라인 곡선과 이 곡선에 대한 볼록 껍질을 보여 준다. 볼록 껍질에 의해 영역이 제한되는 스플라인 곡선은 이 볼록 껍질



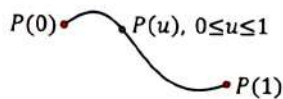
[그림 8-11] 스플라인 곡선과 볼록 껍질

을 통해 크거나 공간상에 존재하는 범위를 개략적으로 알 수 있다. 이는 뷰잉이나 클리핑 등을 할 때 유용하게 활용될 수 있는 정보이다.

스플라인 곡선을 구성하는 각 점의 좌표는 매개변수 u 에 대한 다항식으로 표현된다. 즉, 스플라인 곡선 위의 한 점 $P(u)$ 의 좌표 (x, y, z) 는 식 (8-5)와 같다.

$$x = x(u), y = y(u), z = z(u), \quad 0 \leq u \leq 1 \quad (8-5)$$

여기서 $x(u)$, $y(u)$, $z(u)$ 는 각각 매개변수 u 의 다항식이며, $u=0$ 에 의해 계산되는 좌표가 곡선의 한쪽 끝점이고, $u=1$ 에 의해 계산되는 좌표가 곡선의 다른 쪽 끝점이다(그림 8-12 참조).



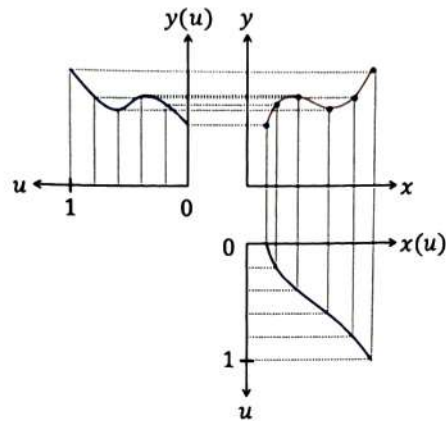
[그림 8-12] 매개변수 스플라인 곡선($0 \leq u \leq 1$)

이 다항식이 n 차 다항식인 경우 이 다항식에 의해 정의되는 스플라인 곡선을 n 차 스플라인 곡선이라고 한다. 그래픽 객체를 설계하는 데 주로 사용되는 것은 3차 스플라인 곡선으로, 식 (8-6)과 같은 매개변수 다항식에 의해 정의된다.

$$\begin{aligned} x(u) &= a_x u^3 + b_x u^2 + c_x u + d_x \\ y(u) &= a_y u^3 + b_y u^2 + c_y u + d_y \\ z(u) &= a_z u^3 + b_z u^2 + c_z u + d_z, \quad 0 \leq u \leq 1 \end{aligned} \quad (8-6)$$

식 (8-6)에서 $x(u)$, $y(u)$, $z(u)$ 는 각각 4개씩의 미지수 (a, b, c, d)가 있으며, 곡선을 정의하기 위한 제어점 및 기타 정보를 이용하여 이 값들을 구한 후 u 를 0부터 1까지 변화시켜 $x(u)$, $y(u)$, $z(u)$ 를 구하여 점을 이으면 스플라인 곡선이 디스플레이된다. [그림 8-13]은 이 과정을 2차원 3차 스플라인 곡선을 통해 예시한 것이다.

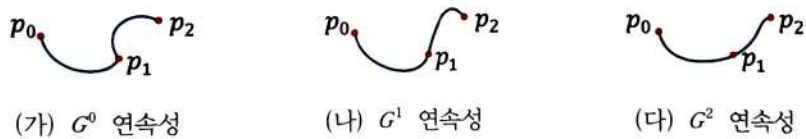
임의의 곡선은 이러한 스플라인 곡선 조각을 이어서 만든다. 이때, 하나의 곡선 조각과 다음 곡선 조각의 연결이 얼마나 부드럽게 이어지는지를 나타내는 것을 연속성이라고 한다. 0차 기하학적 연속성(geometric continuity)은 두 곡선 조각의 끝점이 일치하여



[그림 8-13] 매개변수 다항식으로 정의한 3차 스플라인 곡선

이어지는 것을 의미하며, G^0 연속성이라고 한다. 1차 기하학적 연속성(G^1 연속성)은 G^0 연속성을 만족하면서, 동시에 이어지는 끝점에서 두 곡선의 1차 미분이 비례하는 것을 의미한다. 2차 기하학적 연속성(G^2 연속성)은 G^1 연속성을 만족하면서, 이어지는 끝점에서 2차 미분도 비례하는 것을 의미한다. G^n 연속성($n \geq 1$)은 G^{n-1} 연속성을 만족하면서, 이어지는 끝점에서 n 차 미분이 비례하는 것을 의미한다.

G^0 연속성을 갖는 경우 두 곡선 조각은 끝점이 서로 이어지게 된다. 만일 G^1 연속성을 갖는 경우에는 연결점에서 접선의 방향이 서로 일치하여 연결이 부드럽게 이루어진다. G^2 연속성을 갖는 경우 곡률 중심이 같아진다. [그림 8-14]는 두 스플라인 곡선이 연결되는 끝점 p_1 에서 G^0 , G^1 , G^2 연속성을 가질 경우의 곡선 모양을 보여 준다.



[그림 8-14] 기하학적 연속성(p_1 기준)

때로는 기하학적 형태의 연속성뿐 아니라 매개변수값이 연속적으로 변화할 때, 두 매개변수 곡선의 연결 부분에서 변화의 상대속도를 고려해야 하는 경우도 있다.

이러한 경우에는 매개변수 연속성(parametric continuity)을 사용한다. 0차 매개변수 연속

성(C^0 연속성)은 G^0 연속성과 마찬가지로 단순히 두 곡선 조각이 끝점이 이어지는 것을 의미한다. 1차 매개변수 연속성(C^1 연속성)은 C^0 연속성을 만족하면서 동시에 이어지는 끝점에서 두 곡선의 1차 미분이 동일함을 의미한다. 2차 매개변수 연속성(C^2 연속성)은 C^1 연속성을 만족하면서 동시에 이어지는 끝점에서 2차 미분이 동일함을 의미한다.

C^n 연속성을 만족한다면 G^n 연속성도 만족한다. 그러나 G^n 연속성을 만족한다고 해서 C^n 연속성도 만족되는 것은 아니다. 만일 두 곡선의 연결에 각이 지지 않도록 하는 것이 목표라면 G^1 연속성이면 충분하다. 그러나 좀 더 부드러운 연결을 원한다면 2차 이상의 연속성을 가져야 한다.

연결 부위에서의 상대속도를 고려해야 한다면 매개변수 연속성을 고려해야 한다. G^1 연속성의 경우 접선의 방향은 같지만 크기가 다를 수 있으며, 만일 애니메이션에서 물체나 카메라의 움직임 경로가 G^1 연속성을 갖는 경우 움직임의 속도가 갑자기 변화하는 급격한 움직임의 변화가 발생할 수 있다. 만일 C^1 연속성을 만족한다면 곡선 경계에서 갑자기 속도가 변화되지 않는 부드러운 움직임을 만들 수 있다. 그러나 이 경우에도 가속도의 갑작스런 변화가 발생할 수 있으며, C^2 연속성을 갖는 스플라인 곡선을 이용하면 좀 더 부드럽게 이동할 수 있다.

[그림 8-15]는 G^1 연속성과 C^1 연속성을 갖는 스플라인 곡선의 예이다. G^1 연속성을 갖는 곡선의 경우 1차 미분이 더 큰 곡선(p_1, p_2) 쪽으로 잡아당긴 듯한 모습이 나오는 것을 볼 수 있다.

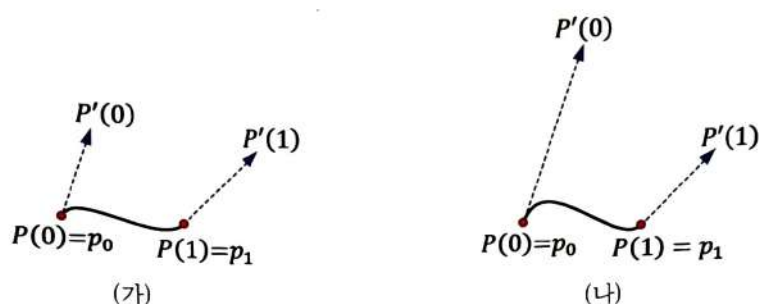


[그림 8-15] 기하학적 연속성과 매개변수 연속성의 비교

8.3.2. 에르미트 스플라인 곡선

하나의 에르미트(Hermite) 스플라인 곡선 조각 P 는 [그림 8-16]과 같이 2개의 제어점 p_0, p_1 과 이 두 점에서의 1차 미분에 의해 정의된다. 이 곡선은 제어점 p_0 와 p_1 을

식 (8-6)과 같은 3차 다항식으로 연결한다. 즉, 매개변수 u 가 0인 경우의 점 $P(0)$ 는 p_0 와 같고, u 가 1인 점 $P(1)$ 은 p_1 과 같다. [그림 8-16] (나)의 경우 (가)에 비해 p_0 에서 P 의 1차 미분 $P'(0)$ 의 크기가 2배이다. 이에 따라 곡선이 $P'(0)$ 방향으로 더 당겨지는 것을 볼 수 있다.



[그림 8-16] 에르미트 스플라인의 정의

에르미트 스플라인 곡선의 계산식을 만들어 보자. $P(u) = [x(u), y(u), z(u)]^T$ 라 할 때, 식 (8-6)은 식 (8-7)과 같은 행렬 표현으로 나타낼 수 있다.

$$P(u) = \begin{bmatrix} a_x & b_x & c_x & d_x \\ a_y & b_y & c_y & d_y \\ a_z & b_z & c_z & d_z \end{bmatrix} \begin{bmatrix} u^3 \\ u^2 \\ u \\ 1 \end{bmatrix} \quad (8-7)$$

곡선을 그리기 위해서는 $P(u)$, $0 \leq u \leq 1$ 을 구해야 하는데, 이를 위해서는 12개의 다항식 계수 $a_x, b_x, c_x, d_x, a_y, b_y, c_y, d_y, a_z, b_z, c_z, d_z$ 를 구해야 한다. 에르미트 스플라인 곡선을 정의하기 위해 사용자가 지정하는 제한 조건 $P(0)$, $P(1)$, $P'(0)$, $P'(1)$ 을 이용하여 풀이한 결과는 식 (8-8)과 같다.

$$\begin{bmatrix} a_x & b_x & c_x & d_x \\ a_y & b_y & c_y & d_y \\ a_z & b_z & c_z & d_z \end{bmatrix} = [P(0) P(1) P'(0) P'(1)] \begin{bmatrix} 2 & -3 & 0 & 1 \\ -2 & 3 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 1 & -1 & 0 & 0 \end{bmatrix} \quad (8-8)$$

그러므로 에르미트 스플라인 곡선 $P(u)$ 는 다음과 같이 구할 수 있다.

$$P(u) = [P(0) \ P(1) \ P'(0) \ P'(1)] \begin{bmatrix} 2 & -3 & 0 & 1 \\ -2 & 3 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 1 & -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} u^3 \\ u^2 \\ u \\ 1 \end{bmatrix} \quad (8-9)$$

여기서

$$M_H = \begin{bmatrix} 2 & -3 & 0 & 1 \\ -2 & 3 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 1 & -1 & 0 & 0 \end{bmatrix} \quad (8-10)$$

을 에르미트 스플라인의 기저행렬(basis matrix)이라고 한다. 또한

$$\begin{bmatrix} H_0(u) \\ H_1(u) \\ H_2(u) \\ H_3(u) \end{bmatrix} = \begin{bmatrix} 2 & -3 & 0 & 1 \\ -2 & 3 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 1 & -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} u^3 \\ u^2 \\ u \\ 1 \end{bmatrix} = \begin{bmatrix} 2u^3 - 3u^2 + 1 \\ -2u^3 + 3u^2 \\ u^3 - 2u^2 + u \\ u^3 - u^2 \end{bmatrix} \quad (8-11)$$

을 에르미트 스플라인의 배합함수(blending function)라고 하며, 이를 식 (8-9)에 대입하여 풀면 식 (8-12)와 같다.

$$\begin{aligned} P(u) &= P(0)H_0(u) + P(1)H_1(u) + P'(0)H_2(u) + P'(1)H_3(u) \\ &= P(0)(2u^3 - 3u^2 + 1) + P(1)(-2u^3 + 3u^2) \\ &\quad + P'(0)(u^3 - 2u^2 + u) + P'(1)(u^3 - u^2) \end{aligned} \quad (8-12)$$

■ 예제 8-1 ■

두 끝점이 $p_0 = (1, 1)$, $p_1 = (5, 3)$ 이고, p_0 에서의 1차 미분이 $(3, 0)$, p_1 에서의 1차 미분이 $(6, -3)$ 인 에르미트 곡선을 그리려고 한다. 매개변수 u 를 0.1씩 변화시키며 곡선상의 좌표 $P(u)$ 를 구하라.

풀이

$P(0)$, $P(1)$, $P'(0)$, $P'(1)$ 의 값

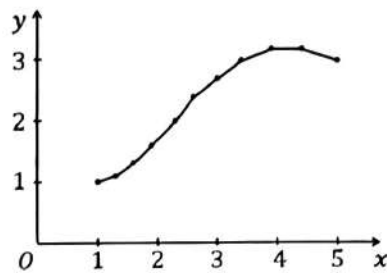
$$P(0) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad P(1) = \begin{bmatrix} 5 \\ 3 \end{bmatrix}, \quad P'(0) = \begin{bmatrix} 3 \\ 0 \end{bmatrix}, \quad P'(1) = \begin{bmatrix} 6 \\ -3 \end{bmatrix}$$

을 식 (8-12)에 넣으면 $P(u)$ 는 다음과 같다.

$$\begin{aligned}
 P(u) &= \begin{bmatrix} 1 \\ 1 \end{bmatrix} (2u^3 - 3u^2 + 1) + \begin{bmatrix} 5 \\ 3 \end{bmatrix} (-2u^3 + 3u^2) \\
 &\quad + \begin{bmatrix} 3 \\ 0 \end{bmatrix} (u^3 - 2u^2 + u) + \begin{bmatrix} 6 \\ -3 \end{bmatrix} (u^3 - u^2) \\
 &= \begin{bmatrix} u^3 + 3u + 1 \\ -7u^3 + 9u^2 + 1 \end{bmatrix}
 \end{aligned}$$

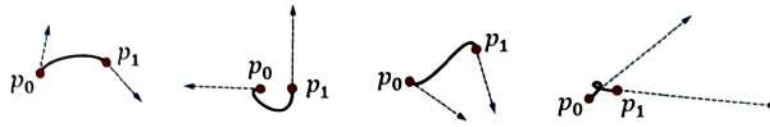
그러므로 곡선상의 좌표는 다음과 같이 계산되며, 이 점들을 차례로 이으면 [그림 8-17]과 같은 결과를 얻을 수 있다.

$P(u)$	u										
	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
$x(u)$	1	1.3	1.6	1.9	2.3	2.6	3.0	3.4	3.9	4.4	5
$y(u)$	1	1.1	1.3	1.6	2.0	2.4	2.7	3.0	3.2	3.2	3

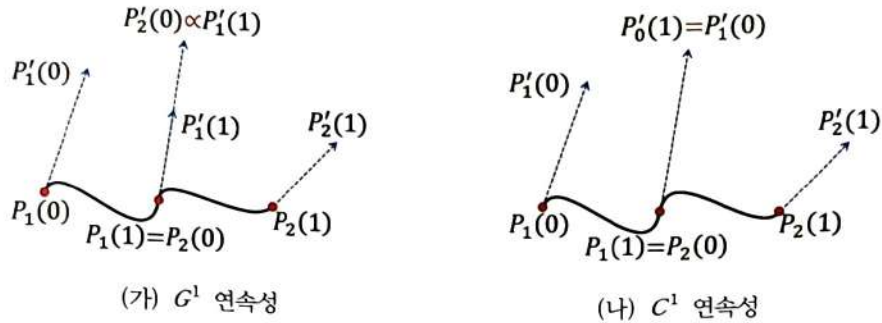


[그림 8-17] 에르미트 스플라인의 예

[그림 8-18]은 여러 가지 형태로 제시된 제한요소에 따라 정의되는 에르미트 곡선의 예이다. 에르미트 곡선을 연속적으로 연결하면 다양한 곡선을 만들 수 있다. 이러한 에르미트 곡선은 끝점 및 1차 미분을 지정하기에 따라 G^0 , G^1 , C^1 연속성을 갖게 된다. 만일 두 곡선이 이어지기만 하면 G^0 연속성을 갖게 될 것이고, 연결되는 끝점에서 1차 미분 벡터의 방향이 같다면 G^1 연속성, 1차 미분 벡터의 방향과 크기가 모두 같다면 C^1 연속성을 만족한다. [그림 8-19]는 G^1 연속성 및 C^1 연속성을 만족하도록 2개의 에르미트 곡선을 그린 예이다.



[그림 8-18] 여러 가지 에르미트 스플라인 곡선



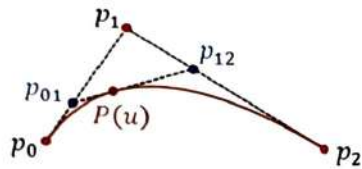
[그림 8-19] 에르미트 스플라인 곡선의 연속성

8.3.3. 베지어 스플라인 곡선

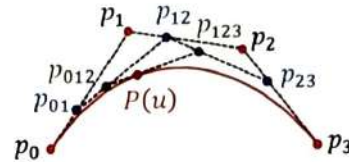
베지어(Bézier) 곡선은 $n+1$ 개의 제어점으로 정의되는 n 차 스플라인 곡선으로, 제어점들을 선형 조합하여 곡선을 형성한다. [그림 8-20]의 (가)는 3개의 제어점 p_0 , p_1 , p_2 에 의해 정의되는 2차 베지어 스플라인 곡선이다.

매개변수 u 에 대해 p_{01} 은 p_0 에 $(1-u)$, p_1 에 u 의 가중치를 주어 선형 조합한 점이고, p_{12} 는 p_1 에 $(1-u)$, p_2 에 u 의 가중치를 주어 선형 조합한 점이다. 스플라인상의 점 $P(u)$ 는 p_{01} 과 p_{12} 를 각각 $(1-u)$ 와 u 의 가중치로 선형 조합한 결과이다. 그러므로 $P(u)$ 는 식 (8-13)과 같이 구할 수 있다.

$$\begin{aligned} p_{01} &= (1-u)p_0 + up_1 \\ p_{12} &= (1-u)p_1 + up_2 \\ P(u) &= (1-u)p_{01} + up_{12} \\ &= (1-u)^2p_0 + 2u(1-u)p_1 + u^2p_2 \end{aligned} \quad (8-13)$$



(가) 2차 베지어 스플라인 곡선



(나) 3차 베지어 스플라인 곡선

[그림 8-20] 베지어 스플라인 곡선의 정의

[그림 8-20]의 (나)는 (가)와 같은 방법으로 4개의 제어점 p_0, p_1, p_2, p_3 에 의해 정의되는 3차 베지어 스플라인 곡선이다. p_0 와 p_1 을 선형 조합하여 p_{01} 을, p_1 과 p_2 를 선형 조합하여 p_{12} 를, p_2 와 p_3 를 선형 조합하여 p_{23} 를 구한다. 그리고 p_{01} 과 p_{12} 를 선형 조합하여 p_{012} 를, p_{12} 와 p_{23} 를 선형 조합하여 p_{123} 를 구한다. 마지막으로 p_{012} 와 p_{123} 를 선형 조합하여 $P(u)$ 를 구하며, 결과는 식 (8-14)와 같다.

$$P(u) = (1-u)^3 p_0 + 3(1-u)^2 u p_1 + 3(1-u) u^2 p_2 + u^3 p_3 \quad (8-14)$$

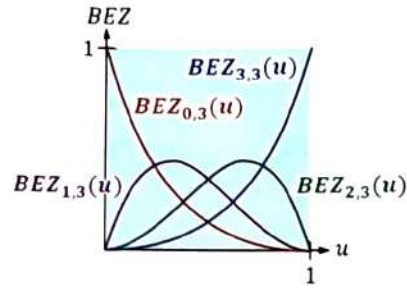
이를 $n+1$ 개의 제어점 $p_0, p_1, p_2, \dots, p_n$ 에 의해 정의되는 n 차 베지어 스플라인 곡선으로 일반화하면 식 (8-15)와 같다.

$$P(u) = \sum_{k=0}^n p_k \text{BEZ}_{k,n}(u), \quad 0 \leq u \leq 1 \quad (8-15)$$

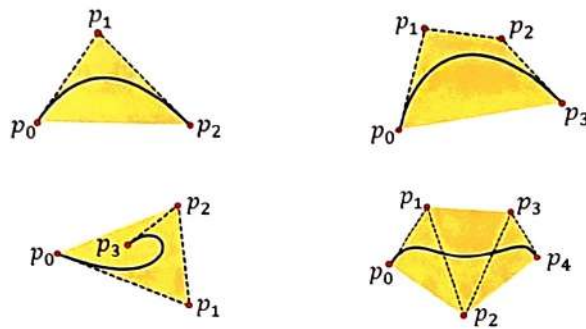
여기서 베지어 스플라인 곡선의 배합함수인 $\text{BEZ}_{k,n}(u)$ 는 식 (8-16)과 같은 번스타인(Bernstein) 다항식이다.

$$\begin{aligned} \text{BEZ}_{k,n}(u) &= C(n,k) u^k (1-u)^{n-k} \\ C(n,k) &= \frac{n!}{k! (n-k)!} \end{aligned} \quad (8-16)$$

[그림 8-21]은 $n=3$ 인 3차 베지어 스플라인 곡선의 배합함수이다. $0 \leq u \leq 1$ 에서 배합함수들의 합은 1이며, $0 < u < 1$ 에서 모든 배합함수는 값이 0이 아니다. 이는 각 제어점 위치가 곡선의 전 구간에서 곡선의 모양에 영향을 미침을 의미한다.



[그림 8-21] 3차 베지어 스플라인 곡선의 배합함수



[그림 8-22] 여러 가지 베지어 스플라인 곡선

[그림 8-22]는 3개, 4개, 5개의 제어점으로 정의되는 베지어 스플라인 곡선의 예이다. 그림에서 볼 수 있듯이 베지어 곡선은 제어점들을 통과하지 않는 근사곡선이나, 처음과 마지막 제어점을 잇는 특성이 있다. 즉, $n+1$ 개의 제어점 $p_0, p_1, p_2, \dots, p_n$ 에 의해 정의되는 베지어 스플라인 곡선의 경우

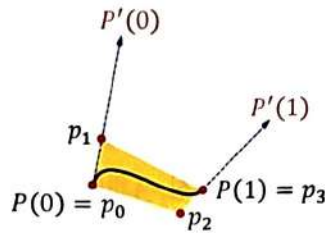
$$P(0) = p_0, \quad P(1) = p_n \quad (8-17)$$

이다. 또한 첫 번째 제어점 p_0 에서의 1차 미분은 $n(p_1 - p_0)$ 와 같고, 마지막 제어점인 p_n 에서의 1차 미분은 $n(p_n - p_{n-1})$ 과 같다. 즉,

$$P'(0) = n(p_1 - p_0), \quad P'(1) = n(p_n - p_{n-1}) \quad (8-18)$$

이다. 그러므로 4개의 제어점 p_0, p_1, p_2, p_3 에 의해 정의되는 3차 베지어 스플라인 곡

선은 [그림 8-23]과 같이 2개의 제어점 p_0, p_3 와 양 끝점에서의 1차 미분이 각각 $3(p_1 - p_0)$ 와 $3(p_3 - p_2)$ 인 에르미트 스플라인 곡선과 동일하다.



[그림 8-23] 에르미트 스플라인과 3차 베지어 스플라인 곡선

■ 예제 8-2 ■

두 끝점이 $p_0 = (1, 2)$, $p_1 = (7, 4)$ 이고, p_0 에서의 1차 미분이 $(6, 3)$, p_1 에서의 1차 미분이 $(-3, 4.5)$ 인 에르미트 곡선이 있다. 이와 동일한 베지어 곡선을 정의하고자 할 때, 제어점은 무엇인가?

풀이

에르미트 곡선은 4개의 제어점으로 정의되는 3차 베지어 스플라인 곡선과 동일하다. 베지어 곡선의 제어점을 q_0, q_1, q_2, q_3 라 하고, p_0 에서의 1차 미분을 $P'(0)$, p_1 에서의 1차 미분을 $P'(1)$ 이라 한다면, 이들 간의 관계는 다음과 같다.

$$q_0 = p_0, \quad q_3 = p_1$$

$$P'(0) = 3(q_1 - q_0), \quad P'(1) = 3(q_3 - q_2)$$

그러므로 q_1 과 q_2 는 각각 다음과 같다.

$$q_1 = p_0 + P'(0)/3$$

$$q_2 = p_1 - P'(1)/3$$

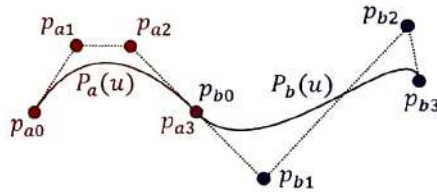
에르미트 곡선의 제한요소를 모두 대입하면 베지어 곡선의 제어점 q_0, q_1, q_2, q_3 는 다음과 같다.

$$q_0 = (1, 2), \quad q_1 = (3, 3), \quad q_2 = (8, 2.5), \quad q_3 = (7, 4)$$

베지어 스플라인 곡선은 제어점들로 형성되는 볼록 껍질 내부에 존재한다. 만일 모든 제어점이 일직선상에 존재한다면 볼록 껍질은 결국 선분으로 축퇴될 것이며, 그 결과 정의되는 베지어 곡선은 볼록 껍질을 벗어날 수 없으므로 선분이 된다.

$n+1$ 개의 제어점에 의해 정의되는 베지어 곡선은 n 차 다항식으로 정의되는 n 차 스플라인이다. 만일 제어점 중 어느 하나의 위치를 변경하면, 제어점 주변의 곡선 일부만 영향을 받는 것이 아니라 곡선 전체가 영향을 받는다. 즉, 곡선의 국부 제어가 되지 않는다. 이러한 점은 곡선을 설계하는 것을 어렵게 한다. 따라서 많은 수의 제어점을 사용하는 고차 베지어 곡선을 설계하는 대신, 4개의 제어점을 갖는 3차 스플라인 곡선 조각을 여러 개 이어서 곡선을 설계하는 방식을 일반적으로 많이 활용한다.

[그림 8-24]는 제어점 $p_{a0}, p_{a1}, p_{a2}, p_{a3}$ 에 의해 정의되는 베지어 곡선 P_a 와 $p_{b0}, p_{b1}, p_{b2}, p_{b3}$ 에 의해 정의되는 베지어 곡선 P_b 를 이어서 만든 곡선이다. p_{a3} 와 p_{b0} 를 동일한 좌표로 지정하면 G^0 연속성, $(p_{a3} - p_{a2}) = m(p_{b1} - p_{b0})$, $m > 0$ 이면 G^1 연속성, $(p_{a3} - p_{a2}) = (p_{b1} - p_{b0})$ 이면 C^1 연속성을 갖는다.



[그림 8-24] 3차 베지어 스플라인 곡선 조각을 이용한 곡선 설계

베지어 곡선은 아핀 불변성(affine invariance)을 갖는다. 즉, 어떠한 베지어 곡선을 아핀 변환¹⁾한 것은 그 곡선의 제어점들을 아핀 변환한 후 그 제어점들로 만들어지는 베

1) 아핀 변환은 다음과 같은 변환이 이루어지는 변환으로, 평행선은 변환 후 평행선에 대응되고, 유한공간의 점은 역시 유한공간의 점으로 변환된다. 이동·회전·크기조정·대칭·밀립변환은 아핀 변환의 대표적인 예이다.

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{xx} & a_{xy} & a_{xz} & b_x \\ a_{yx} & a_{yy} & a_{yz} & b_y \\ a_{zx} & a_{zy} & a_{zz} & b_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

지어 곡선과 일치한다는 것이다. 변환에 대한 불변성은 투영과 관련하여 중요하다. 평행투영은 아핀 변환에 의해 투영이 이루어지기 때문에 베지어 곡선은 평행투영에 대해 불변성이 성립한다. 그러나 원근투영은 아핀 변환이 아니며, 투영된 제어점으로 정의되는 곡선은 원래의 곡선을 투영한 결과와 일치하지 않을 수 있다.

8.3.4. B-스플라인 곡선

B-스플라인 곡선은 CAD를 비롯한 많은 그래픽스 패키지에서 널리 제공되는 스플라인 곡선이다. 베지어 곡선의 차수가 제어점의 수에 의해 결정되는 것과 달리, B-스플라인의 배합함수는 곡선 설계자가 차수를 결정할 수 있으며, 국부 제어가 가능하다는 특성이 있다. 또한 d 차 B-스플라인은 C^{d-1} 연속성을 갖는 곡선을 생성할 수 있다. 흔히 사용되는 3차 B-스플라인은 C^2 연속성을 가질 수 있어 C^1 연속성을 제공할 수 있는 베지어 스플라인보다 부드럽게 변화하는 곡선을 설계할 수 있다.

에르미트 곡선이나 베지어 곡선과 같은 매개변수 다항식으로 표현되는 스플라인에서 매개변수 u 는 0부터 1 사이의 값을 갖는 것이었으나, B-스플라인에서는 곡선 설계자가 이를 정한다. 이때, 매개변수의 값을 여러 개의 구간으로 나누어 정한다. $n+1$ 개의 제어점 $p_0, p_1, p_2, \dots, p_n$ 이 주어질 때, d 차 다항식으로 표현되는 B-스플라인 곡선은 $d+n+1$ 개의 매개변수 구간으로 설계한다. 식 (8-19)는 이와 같이 설계되는 d 차 B-스플라인 곡선의 방정식이다.

$$P(u) = \sum_{k=0}^n p_k B_{k,d}(u), \quad u_{\min} \leq u \leq u_{\max}, \quad 1 \leq d \leq n \quad (8-19)$$

이때, 배합함수 B 는 다음과 같은 콕스-드보어(Cox-de Boor) 점화식으로 정의된다.

$$B_{k,0}(u) = \begin{cases} 1, & \text{if } u_k \leq u < u_{k+1} \\ 0, & \text{otherwise} \end{cases}$$

$$B_{k,d}(u) = \frac{u - u_k}{u_{k+d} - u_k} B_{k,d-1}(u) + \frac{u_{k+d+1} - u}{u_{k+d+1} - u_{k+1}} B_{k+1,d-1}(u) \quad (8-20)$$

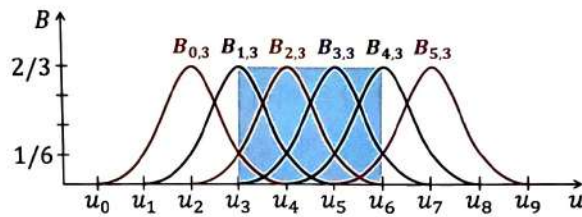
식 (8-20)을 순환적으로 적용해 보면 $B_{k,0}(u)$ 는 매개변수 구간 $u_k \leq u < u_{k+1}$ 범위에 0이 아닌 값을 갖고, $B_{k,1}(u)$ 는 매개변수 구간 $u_k \leq u < u_{k+2}$ 범위에서,

$B_{k,d}(u)$ 는 매개변수 구간 $u_k \leq u < u_{k+d+1}$ 범위에서 0이 아닌 값을 갖는다. 이때, 각 구간의 경계에 해당되는 $u_0, u_1, u_2, \dots$ 등을 매듭(knot)이라고 한다.

(1) 균일 B-스플라인

매듭의 간격이 동일한 경우 균일(uniform) B-스플라인이라고 한다. 예를 들어, 매듭 값이 모두 1의 간격으로 나열된다면, 즉 매듭 벡터가 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10) 이면 이에 의해 정의되는 B-스플라인은 균일 B-스플라인이다. 매듭의 시작값이나 간격은 의미가 없으며, 이처럼 0부터 시작하여 1의 간격으로 나열되는 형태의 매듭 벡터가 편리하게 사용된다. 또는 (0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0)과 같이 전체 매듭값의 범위를 0부터 1 사이의 값으로 정규화하여 사용하기도 한다.

[그림 8-25]는 균일 3차 B-스플라인의 배합함수를 보여 주는 그림이다. 여기서는 모든 배합함수가 동일하며, 배합함수는 동일한 간격으로 나열된다.



[그림 8-25] 균일 3차 B-스플라인의 배합함수(■: 곡선이 정의되는 구간)

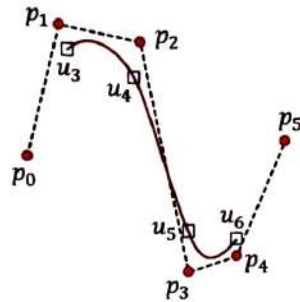
매개변수의 범위 $u_d \leq u < u_{n+1}$ (6개의 제어점으로 정의되는 3차 B-스플라인의 경우 $u_3 \leq u < u_6$ 의 범위)에서 배합함수의 합은 1이다.

$$\sum_{k=0}^n B_{k,d}(u) = 1, \quad u_d \leq u < u_{n+1} \quad (8-21)$$

이는 매개변수의 범위 $u_d \leq u < u_{n+1}$ 내에서 B-스플라인에 의해 정의되는 곡선이 제어점들에 의해 형성되는 볼록 껍질 내부에 포함된다는 것을 의미한다. 이때, 매듭 $u_{d+1}, u_{d+2}, \dots, u_n$ 을 내부 매듭(internal knots)이라고 한다.

[그림 8-26]은 6개의 제어점 $p_0, p_1, p_2, p_3, p_4, p_5$ 와 10개의 값으로 구성된 매듭

벡터 $(u_0, u_1, u_2, u_3, u_4, u_5, u_6, u_7, u_8, u_9) = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)$ 에 의해 정의되는 균일 3차 B-스플라인 곡선이다. [그림 8-25]에서 매듭값 u_3 부터 u_4 사이에서는 $B_{0,3}(u)$, $B_{1,3}(u)$, $B_{2,3}(u)$, $B_{3,3}(u)$ 만 0이 아닌 값을 가진다. 그러므로 이 구간의 곡선은 p_0, p_1, p_2, p_3 에 의해 영향을 받으며, p_0, p_1, p_2, p_3 에 의한 블록 꺾질 내부에 포함된다[이를 일반화하면 u_k 부터 u_{k+1} 사이의 곡선은 $p_{k-3}, p_{k-2}, p_{k-1}, p_k$ 에 의한 블록 꺾질 내부에 포함된다($k \geq 3$)].

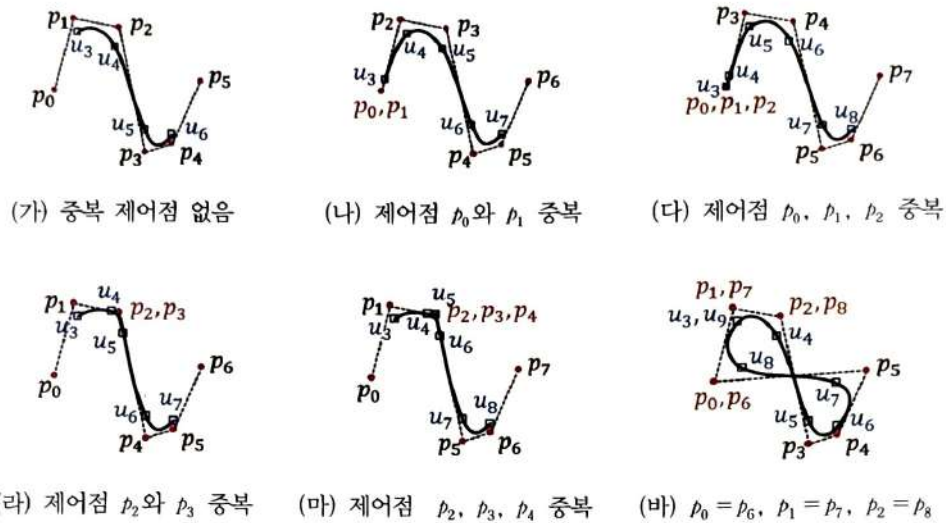


[그림 8-26] 균일 3차 B-스플라인 곡선(●: 제어점, □: 매듭)

역으로 $B_{0,3}(u)$ 는 $u_0 \leq u < u_4$ 구간에서만 0이 아닌 값을 가지므로 p_0 의 위치는 u_3 부터 u_4 사이의 곡선 구간의 모양에만 기여하고, 그 이후의 곡선 구간에는 영향을 미치지 않는다. 마찬가지로 p_1 은 u_3 부터 u_5 사이의 곡선 구간, p_2 와 p_3 는 u_3 부터 u_6 사이의 곡선 구간, p_4 는 u_4 부터 u_6 , p_5 는 u_5 부터 u_6 사이의 곡선 구간의 모양에만 영향을 미친다.

[그림 8-26]에서 볼 수 있는 것처럼 균일 B-스플라인 곡선은 제어점을 지나지 않는 근사곡선이다. 제어점을 같은 곳에 중복하여 나열하면 원하는 점을 통과하도록 곡선을 제어할 수 있다. [그림 8-27]의 (나)에서는 제어점 p_0 와 p_1 이 같은 위치에 중복되어 있다. 이 경우 곡선의 시작점인 매듭 u_3 에서의 좌표는 p_0 와 p_1 방향으로 접근한 것을 볼 수 있다. 만일 (다)와 같이 세 제어점 p_0, p_1, p_2 가 동일한 위치에 중복된다면 매듭 u_3 에서의 좌표는 p_0, p_1, p_2 와 같은 위치가 된다.

이런 곡선 제어는 [그림 8-25]를 통해 알 수 있다. 즉, $B_{0,3}(u_3) = B_{2,3}(u_3) = 1/6$, $B_{1,3}(u_3) = 4/6$, $B_{3,3}(u_3) = 0$ 이므로, u_3 에서의 곡선 좌표는 $(p_0 + 4p_1 + p_2)/6$ 이다.



[그림 8-27] 제어점 중복에 의한 균일 3차 B-스플라인 곡선(●: 제어점, □: 매듭)

만일 [그림 8-27]의 (다)에서와 같이 p_0, p_1, p_2 가 같은 점이라면 이 좌표의 값은 p_0 (또는 p_1, p_2)가 된다. 그러므로 정의되는 곡선은 p_0 를 시작점으로 하는 곡선이 된다.

이와 같이 제어점을 중복하면 곡선은 중복 제어점 방향으로 당겨진다. [그림 8-27]의 (라), (마) 역시 이러한 관계를 보여 준다. 다만 이 경우 곡선의 연속성은 낮아지며, 곡선이 통과하게 되는 3중 제어점(그림 8-27 (마)의 p_2, p_3, p_4)에서는 C^0 연속성만 갖게 된다.

제어점을 중복함으로써 폐곡선을 만들 수도 있다. 이를 위해서는 처음 3개의 제어점을 마지막에 반복하여 나열한다. 즉, [그림 8-27]의 (바)와 같이 제어점이 $p_0, p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8$ 일 때, $p_0 = p_6, p_1 = p_7, p_2 = p_8$ 으로 하는 것이다. 이 경우 u_3 와 u_9 에서의 곡선좌표가 동일하며, 그 결과 폐곡선이 만들어진다.

(2) 비균일 B-스플라인

B-스플라인 곡선의 형태를 제어하는 다른 방법은 매듭 벡터를 이용하는 것이다. 이 방법은 매듭값 사이의 간격이 일정한 균일 B-스플라인과 달리 매듭 사이의 간격을 변화시킴으로써 곡선의 모양을 제어할 수 있다. 균일하지 않은 매듭 간격은 특정 배합함수의 폭이 좁아지면서 최댓값이 커지게 하며, 이와 관련된 제어점으로 곡선을 끌어당

기는 효과가 있다. 이처럼 매듭값의 간격이 균일하지 않은 B-스플라인을 비균일(nonuniform) B-스플라인이라고 한다.

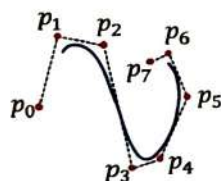
비균일 B-스플라인을 활용하는 일반적인 방법은 매듭값을 중복하는 것이다. 매듭값을 중복하면 관련된 제어점에 가깝게 곡선이 형성된다. [그림 8-28]의 (가)는 8개의 제어점과 12개의 값으로 구성된 매듭 벡터 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)에 의해 정의되는 균일 3차 B-스플라인이다.

[그림 8-28]의 (나)는 6, 7번째 매듭값이 모두 5인 것을 볼 수 있다. 이 경우 곡선이 p_3 방향으로 당겨진 것을 볼 수 있다. 또한 이와 같이 중복된 매듭에서 연속성이 한 단계 감소하는데, 3차 B-스플라인의 연속성인 C^2 에서 C^1 으로 바뀐다.

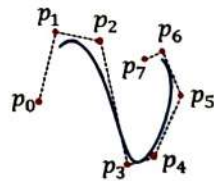
[그림 8-28]의 (다)는 5~7번째의 3개의 매듭이 모두 5라는 중복된 값이 사용된 경우이다. 이 경우 매듭값 5가 제어점 p_3 위에 놓이게 되며(인접한 두 매듭이 네 제어점으로 형성되는 블록 겹칠 내부에 존재해야 하므로), p_3 는 곡선 위에 있게 된다. 또한 연속성도 C^0 으로 바뀌어 부드러운 연결이 아닌 각진 연결이 만들어진다.

[그림 8-28]의 (라)는 5~8번째의 4개의 매듭이 모두 5라는 중복된 값이 사용된 경우이다. 이 경우에는 곡선에 불연속이 발생한다.

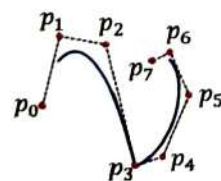
d 차 B-스플라인에서 중복된 매듭을 사용하는 대표적인 예는 앞과 뒤 각각 $d+1$ 개의 매듭값을 동일한 값으로 나열하고, 그 외의 매듭값은 균일한 간격으로 나열하는 것



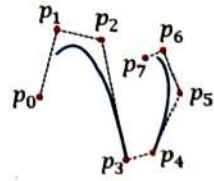
(가) (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)



(나) (0, 1, 2, 3, 4, 5, 5, 7, 8, 9, 10, 11)



(다) (0, 1, 2, 3, 5, 5, 5, 7, 8, 9, 10, 11)



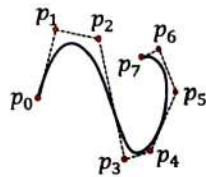
(라) (0, 1, 2, 3, 5, 5, 5, 5, 8, 9, 10, 11)

[그림 8-28] 매듭 중복에 의한 곡선 제어

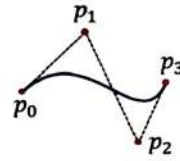
이다. 이와 같이 할 경우 곡선은 첫 번째 제어점과 마지막 제어점을 잇게 된다.

[그림 8-29] (가)의 경우 12개의 매듭값 중 처음 4개는 0, 그다음부터는 1의 간격으로 증가하다가 마지막 4개는 모두 동일한 5라는 값을 매듭값으로 사용하고 있다. 이 경우 그림에서처럼 양 끝점을 잇는 곡선이 만들어진다. 이러한 형태의 B-스플라인을 개방(open) B-스플라인이라고 한다.

[그림 8-29]의 (나)는 4개의 제어점으로 정의되는 개방 3차 B-스플라인 곡선이다. 이 경우 생성되는 곡선은 동일한 제어점으로 만들어지는 베지어 스플라인 곡선과 동일하다.



(가) (0, 0, 0, 0, 1, 2, 3, 4, 5, 5, 5, 5)



(나) (0, 0, 0, 0, 1, 1, 1, 1)

[그림 8-29] 개방 3차 B-스플라인

(3) 비균일 유리 B-스플라인

스플라인을 사용하는 것은 임의의 곡선을 표현하기 위해 하나하나의 점의 좌표를 기억할 필요 없이 몇 개의 제어점만으로 표현할 수 있게 한다. 만일 이러한 곡선을 기하 변환한다면 곡선의 모든 점을 일일이 변환하지 않고 제어점만 변환한 후, 변환된 제어점으로 스플라인 곡선을 만들어도 동일한 결과를 얻을 수 있어야 한다.

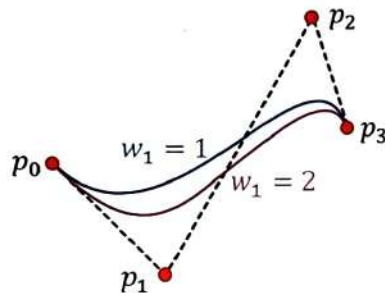
지금까지 소개한 스플라인 곡선들은 아핀 불변성이 성립한다. 그러나 만일 필요한 변환이 아핀 변환이 아니면 변환된 제어점들에 의해 정의되는 곡선은 곡선을 직접 변환한 것과 일치하지 않는다. 특히 3차원 공간에 정의된 객체를 화면에 보여 주기 위해서는 이를 2차원 평면에 투영하는 뷰잉 과정을 거쳐야 하는데, 평행투영은 아핀 변환이지만 원근투영은 아핀 변환이 아니므로 변환에 따른 불변성이 성립하지 않는 문제가 있다.

비균일 유리 B-스플라인(NonUniform Rational B-Spline: NURBS)은 아핀 변환뿐 아니라 원근투영에 대해서도 불변성이 성립하는 유용한 스플라인이다. 여기서 '유리'는 배

합함수가 유리함수(rational function)임을 나타낸다. NURBS는 스플라인의 차수, 가중치를 포함하는 제어점 리스트, 그리고 비균일 매듭 벡터로 정의된다. 식 (8-22)는 NURBS의 방정식이다.

$$P(u) = \frac{\sum_{k=0}^n w_k p_k B_{k,d}(u)}{\sum_{k=0}^n w_k B_{k,d}(u)} \quad (8-22)$$

여기서 w_k 는 제어점 p_k 에 대한 가중치로, w_k 가 클수록 p_k 가 영향력을 크게 미쳐 곡선이 p_k 에 가깝게 끌린다. 이러한 결과는 [그림 8-30]을 통해 볼 수 있다.



[그림 8-30] 제어점 가중치에 따른 NURBS의 변화
[매듭 벡터는 (0, 0, 0, 0, 1, 1, 1, 1)이며, $w_0 = w_2 = w_3 = 1$]

모든 가중치를 1로 지정하면 식 (8-22)는 식 (8-19)와 동일하다. 더불어 [그림 8-30]의 예에서처럼 가중치가 모두 1인 4개의 제어점과 시작과 끝이 중복된 값으로 구성된 비균일 매듭 벡터를 사용할 경우 그 결과는 베지어 곡선과 동일하다.

NURBS 곡선이 갖는 또 하나의 유용한 특성은 타원, 포물선, 쌍곡선 등의 원뿔곡선(conic section)과 같은 표준곡선을 표현할 수 있다는 것이다. 비유리 B-스플라인을 이용하면 원뿔곡선을 근사화할 수 있을 뿐이다. 원뿔곡선은 3개의 제어점 p_0, p_1, p_2 와 각 제어점의 가중치 $w_0 = 1, w_1 = r/(1-r), w_2 = 1$, 매듭 벡터 (0, 0, 0, 1, 1, 1)으

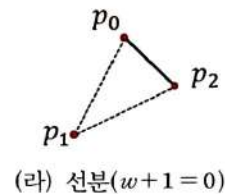
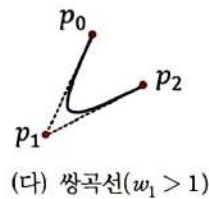
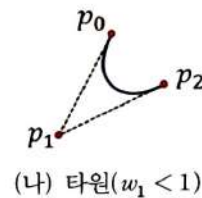
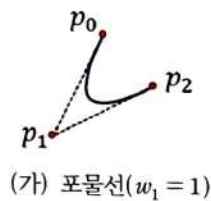
2) 유리함수란 두 다항식의 비율로 표현되는 함수를 의미한다. 즉, $g(x)$ 와 $h(x)$ 가 다항식일 때 $f(x) = \frac{g(x)}{h(x)}$ 형태의 함수 $f(x)$ 를 유리함수라 한다. $f(x) = \frac{x^2 - 2x - 1}{x^2 + 2x + 3}$ 은 유리함수의 예이다.

로 정의되는 2차 NURBS 곡선으로 표현할 수 있다. 이때, r 은 $0 \leq r < 1$ 이며, r 의 값에 따라 [그림 8-31]과 같은 원뿔곡선이 된다. $r = 0$ 이면 선분으로 축퇴된다.

$$r = 0.5 \quad (w_1 = 1) \rightarrow \text{포물선}$$

$$r < 0.5 \quad (w_1 < 1) \rightarrow \text{타원}$$

$$r > 0.5 \quad (w_1 > 1) \rightarrow \text{쌍곡선}$$



[그림 8-31] 원뿔곡선을 표현하는 NURBS 곡선
[매듭 벡터는 $(0, 0, 0, 1, 1, 1)$ 이며, $w_0 = w_2 = 1$]

8.3.5. 스플라인 곡선의 특성 비교

스플라인 곡선은 주어진 제어점들을 연결하거나 근사화하는 부드러운 곡선을 얻기 위해 사용된다. 이러한 곡선의 특성은 얼마나 부드러운 곡선을 만들 수 있는지, 제어점을 연결할 수 있는지, 볼록 껍질 내부에 곡선이 들어가는지, 국부 제어가 가능한지, 변환에 따른 불변성을 갖는지 등 여러 가지 관점에서 비교해 볼 수 있다.

에르미트 곡선이나 베지어 곡선은 여러 개의 스플라인 곡선을 연속적으로 이어 붙여 임의의 자유곡선을 표현할 수 있으며, 이때 연결 부위에서 G^0 나 G^1 , C^1 연속성을 갖도록 제어할 수 있다. 3차 B-스플라인은 G^2 , C^2 연속성을 갖는 곡선을 만들 수 있다. C^2 연속성은 2차 미분이 일치하는 것이므로 움직임의 가속도가 연속적으로 변화할 수

있도록 한다. 그러므로 애니메이션 시퀀스의 움직임 경로가 C^1 연속성인 경우에 비해 C^2 연속성을 갖는 경우 이동 경로를 더욱 부드럽게 만들 수 있다.

에르미트 곡선은 두 제어점을 잇는 스플라인 곡선이다. 베지어 곡선은 처음과 마지막 제어점을 잇고, 나머지 제어점은 곡선의 형태를 제어하는 역할을 한다. B-스플라인은 제어점을 근사화하는 곡선이지만, 제어점의 위치를 중복하거나 여러 개의 매듭값을 중복함으로써 특정 제어점을 통과하도록 할 수 있다. 단, 이 경우 연속성은 낮아진다.

베지어 곡선은 식 (8-15)와 (8-16)을 통해 알 수 있는 것처럼 각 제어점이 전체 곡선 구간에 영향을 미치므로 국부 제어를 할 수 없다. 대신 전체 곡선을 몇 개의 곡선의 연결 형태로 만들 수 있으며, 각 부분 곡선 구간 단위로 곡선을 제어하는 방법을 사용할 수 있다. 이때, 이어지는 곡선 구간의 연결이 G^1 연속성을 갖도록 하려면 이전 곡선 구간의 마지막 두 제어점과 다음 곡선 구간의 처음 두 제어점이 직선상에 연속적으로 놓이도록 하면 된다. B-스플라인은 국부 제어가 가능하다. n 차 B-스플라인의 경우 하나의 곡선 구간은 $n+1$ 개의 제어점의 영향을 받는다.

에르미트 곡선, 베지어 곡선, 비유리 B-스플라인은 아핀 불변성을 갖는다. 그러므로 곡선을 이동·회전·크기·밀립변환을 하는 것은 제어점을 변환한 후 변환된 제어점에 의해 정의된 곡선과 동일하다. 평행투영은 아핀 변환에 해당되어 불변성이 성립하나, 원근투영의 경우 아핀 변환이 아니어서 불변성이 성립하지 않는다. 반면에, NURBS 곡선은 원근투영의 경우에도 불변성이 성립한다.

〈표 8-2〉는 스플라인 곡선의 특성을 정리한 것이다.

〈표 8-2〉 스플라인 곡선의 특성

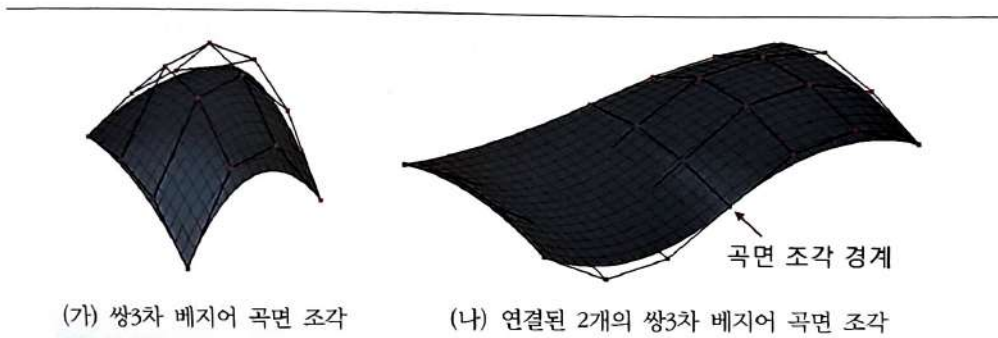
특성 \ 종류	에르미트 곡선	베지어 곡선	3차 비유리 B-스플라인	3차 NURBS
연속성	G^0, G^1, C^1	G^0, G^1, C^1	G^0, G^1, C^1, G^2, C^2	G^0, G^1, C^1, G^2, C^2
일부 제어점 연결	○	○	△	△
모든 제어점 연결	○	×	×	×
국부 제어	N/A	×	○	○
아핀 불변성	○	○	○	○
원근투영 불변성	×	×	×	○
원뿔곡선 재현	×	×	×	○

8.3.6. 스플라인 곡면

2개의 스플라인 곡선집합을 3차원 공간상에서 교차시키면 곡면을 나타낼 수 있다. 베지어 곡면은 2개의 매개변수 u 와 v 에 의해 정의되는 매개변수 곡면이다. 4×4 개의 제어점으로 정의되는 쌍3차 베지어 곡면(bicubic Bézier surface)은 식 (8-23)과 같이 정의된다.

$$P(u, v) = \sum_{j=0}^3 \sum_{k=0}^3 p_{jk} \text{BEZ}_{3,j}(u) \text{BEZ}_{3,k}(v), \quad 0 \leq u, v \leq 1 \quad (8-23)$$

[그림 8-32]의 (가)는 쌍3차 베지어 곡면 조각의 예이다. 여러 개의 쌍3차 베지어 곡면 조각을 이어 붙이면 임의 형태의 표면을 만들 수 있다. 이때, 베지어 곡선에서처럼 이어지는 곡면의 제어점을 공유하게 함으로써 G^0 연속성을 얻을 수 있으며, 공유 제어점을 마주 보는 제어점이 일직선상에 있도록 함으로써 G^1 , C^1 연속성을 얻을 수 있다. (나)는 2개의 쌍3차 베지어 곡면 조각이 C^1 연속성을 갖도록 연결된 것을 보여 준다.



[그림 8-32] 쌍3차 베지어 곡면

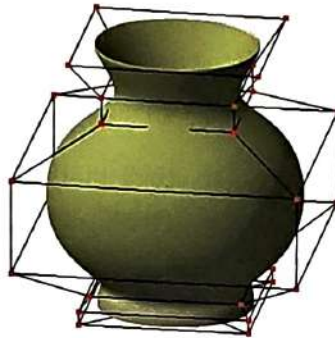
NURBS 곡면 역시 베지어 곡면처럼 배합함수의 데카르트 곱을 이용하여 식 (8-24)와 같이 구할 수 있다.

$$P(u, v) = \frac{\sum_{j=0}^m \sum_{k=0}^n w_{jk} p_{jk} B_{j,d}(u) B_{k,d}(v)}{\sum_{j=0}^m \sum_{k=0}^n w_{jk} B_{j,d}(u) B_{k,d}(v)} \quad (8-24)$$

모든 w_{jk} 의 값이 동일하다면 식 (8-24)는 식 (8-25)와 같이 되며, 이는 비유리 B-스플라인 곡면이다.

$$P(u, v) = \sum_{j=0}^m \sum_{k=0}^n p_{jk} B_{j,d}(u) B_{k,d}(v) \quad (8-25)$$

[그림 8-33]은 NURBS 곡면을 이용하여 그린 화병이다.



[그림 8-33] NURBS 곡면으로 그린 화병

8.3.7. OpenGL의 테셀레이션 셰이더와 스플라인 곡면

OpenGL 4.0부터는 테셀레이션(tessellation) 셰이더를 이용하여 스플라인 곡면을 처리할 수 있다. 테셀레이션은 작은 조각을 이어 붙여 모자이크식으로 꾸미는 것을 말한다. OpenGL의 그래픽스 파이프라인에는 테셀레이션과 관련하여 3개의 단계가 포함된다. 테셀레이션 제어 셰이더(tessellation control shader)와 테셀레이션 계산 셰이더(tessellation evaluation shader)는 셰이더 프로그램에 의해 동작하는 프로그램 가능 단계이고, 그 사이에 존재하는 테셀레이션 기본 요소 발생기(tessellation primitive generator)는 고정기능 단계이다. [소스코드 8-2]는 테셀레이션 셰이더를 이용하여 쌍3차 베지어 스플라인 곡면을 생성한다.

[소스코드 8-2] BezierSurface.c

```

1  #include <stdio.h>
2
3  #include <gl/glew.h>
4  #include <gl/freeglut.h>
5  #include "gTransform.h"
6
7  typedef struct {    // 색을 표현하기 위한 구조체
8      float r, g, b, a;
9  } VecColor4f;
10
11  enum { B_SURFACE, N_VBOs };
12  GLuint VBO[N_VBOs];    // 꼭짓점 버퍼 객체
13  GLuint uGTransLocation;    // 기하변환 행렬 유니폼의 위치
14  GLuint uGColorLocation;    // 색 속성 유니폼의 위치
15
16  static const char* pVS =    // 꼭짓점 셰이더 소스
17  "#version 400 core          \n"
18  "layout (location = 0) in vec3 Position;    \n"
19  "                                           \n"
20  "void main()                  \n"
21  "{                             \n"
22  "    gl_Position = vec4(Position, 1.0);    \n"
23  "};
24
25  static const char* pTCS =    // 테셀레이션 제어 셰이더 소스
26  "#version 400 core          \n"
27  "layout(vertices = 16) out;    \n"
28  "                                           \n"
29  "void main( )                \n"
30  "{                             \n"
31  "    gl_TessLevelInner[0] = 16;    \n"
32  "    gl_TessLevelInner[1] = 16;    \n"
33  "    gl_TessLevelOuter[0] = 16;    \n"

```

```

34     "    gl_TessLevelOuter[1] = 16;                                \n"
35     "    gl_TessLevelOuter[2] = 16;                                \n"
36     "    gl_TessLevelOuter[3] = 16;                                \n"
37     "                                                                \n"
38     "    gl_out[gl_InvocationID].gl_Position =                        \n"
39     "        gl_in[gl_InvocationID].gl_Position;                    \n"
40     "}                                                                \n";
41
42     static const char* pTES =    // 테셀레이션 계산 셰이더 소스
43     "#version 400 core                                                \n"
44     "layout(quads, equal_spacing, ccw) in;                            \n"
45     "uniform mat4 gTransform;                                          \n"
46     "                                                                \n"
47     "float B(int i, float u)                                           \n"
48     "{                                                                \n"
49     "    const vec4 bc = vec4(1, 3, 3, 1);                            \n"
50     "    return bc[i] * pow(u, i) * pow(1.0 - u, 3 - i);              \n"
51     "}                                                                \n"
52     "                                                                \n"
53     "void main()                                                        \n"
54     "{                                                                \n"
55     "    vec4 pos = vec4(0.0);                                          \n"
56     "    vec4 col = vec4(0.0);                                          \n"
57     "    float u = gl_TessCoord.x;                                       \n"
58     "    float v = gl_TessCoord.y;                                       \n"
59     "                                                                \n"
60     "    for (int k = 0; k < 4; ++k) {                                    \n"
61     "        for (int j = 0; j < 4; ++j) {                              \n"
62     "            pos += B(j, u) * B(k, v) * gl_in[4 * k + j].gl_Position; \n"
63     "        }                                                            \n"
64     "    }                                                                \n"
65     "    gl_Position = gTransform * pos;                                  \n"
66     "}                                                                \n";
67

```

```

68 static const char* pFS = // 프래그먼트 셰이더 소스
69 "#version 400 core \n"
70 "out vec4 FragColor; \n"
71 "uniform vec4 gColor; \n"
72 " \n"
73 "void main( ) \n"
74 "{ \n"
75 "    FragColor = gColor; \n"
76 "}"; \n"
77
78 void AddShader(GLuint shaderProg, const char* pShaderSrc,
               GLint ShaderType)
79 { // 셰이더 생성
80     GLuint shader = glCreateShader(ShaderType);
81     if (!shader) {
82         fprintf(stderr, "오류 - Shader 생성(%d)\n", ShaderType);
83         exit(0);
84     }
85     // 셰이더 컴파일
86     const GLchar* src[1] = { pShaderSrc };
87     const GLint len[1] = { strlen(pShaderSrc) };
88     glShaderSource(shader, 1, src, len);
89     glCompileShader(shader);
90     GLint success;
91     glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
92     if (!success) { // 컴파일 오류 발생
93         GLchar infoLog[256];
94         glGetShaderInfoLog(shader, 256, NULL, infoLog);
95         fprintf(stderr, "오류 - Shader 컴파일(%d): %s\n",
96                 ShaderType, infoLog);
97         exit(1);
98     }
99     // 셰이더 프로그램에 컴파일된 셰이더를 추가
    glAttachShader(shaderProg, shader);

```

```

100     }
101
102     void SetUpShaders( )
103     { // 셰이더 프로그램 객체 생성
104         GLuint shaderProg = glCreateProgram( );
105         if (!shaderProg) {
106             fprintf(stderr, "오류 - Shader 프로그램 생성\n");
107             exit(1);
108         }
109
110         // 꼭짓점 셰이더 및 프래그먼트 셰이더 적재
111         AddShader(shaderProg, pVS, GL_VERTEX_SHADER);
112         AddShader(shaderProg, pTCS, GL_TESS_CONTROL_SHADER);
113         AddShader(shaderProg, pTES, GL_TESS_EVALUATION_SHADER);
114         AddShader(shaderProg, pFS, GL_FRAGMENT_SHADER);
115
116         GLint success = 0;
117         GLchar errLog[256];
118
119         glLinkProgram(shaderProg); // 셰이더 프로그램 링크
120         glGetProgramiv(shaderProg, GL_LINK_STATUS, &success);
121         if (!success) {
122             glGetProgramInfoLog(shaderProg, sizeof(errLog), NULL, errLog);
123             fprintf(stderr, "오류 - Shader 프로그램 링크: %s\n", errLog);
124             exit(1);
125         }
126
127         glValidateProgram(shaderProg); // 프로그램 객체가 유효한지 검사
128         glGetProgramiv(shaderProg, GL_VALIDATE_STATUS, &success);
129         if (!success) {
130             glGetProgramInfoLog(shaderProg, sizeof(errLog), NULL, errLog);
131             fprintf(stderr, "Invalid shader program: %s\n", errLog);
132             exit(1);
133         }

```

```

134     glUseProgram(shaderProg);           // 현재 셰이더 프로그램 객체로 지정
135     uGTransLocation = glGetUniformLocation(shaderProg, "gTransform");
136     uGColorLocation = glGetUniformLocation(shaderProg, "gColor");
137
138     if (uGTransLocation == 0xFFFFFFFF || uGColorLocation == 0xFFFFFFFF) {
139         fprintf(stderr, "Uniform location not found!!\n");
140         exit(1);
141     }
142 }
143
144 void RenderCB( )
145 {
146     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
147     glClearDepth(1.0);
148     glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT); // 화면과 깊이버퍼 지움
149
150     TransMat4f viewMat;
151     Vec3f pOrg = { 1.5f, 2.0f, 2.5f };
152     Vec3f pRef = { 0.0f, 0.0f, 0.0f };
153     Vec3f vUp = { 0.0f, 0.0f, 1.0f };
154     viewingTrans(&viewMat, &pOrg, &pRef, &vUp);
155
156     TransMat4f orthoMat;
157     orthogonalPrj(&orthoMat, -3.0f, 3.0f, -3.0f, 3.0f, -3.0f, 3.0f);
158
159     TransMat4f tMat= orthoMat;
160     matmul_GTM(&tMat, &viewMat);
161     glUniformMatrix4fv(uGTransLocation, 1, GL_TRUE, tMat.mat);
162
163     glEnableVertexAttribArray(0);
164     glBindBuffer(GL_ARRAY_BUFFER, VBO[B_SURFACE]);
165     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
166     VecColor4f col = { 0.0f, 1.0f, 1.0f, 1.0f };
167     glUniform4fv(uGColorLocation, 1, &col.r);

```

```

168     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
169     glDrawArrays(GL_PATCHES, 0, 16);
170     col.r = col.g = col.b = 0.0f;
171     glUniform4fv(uGColorLocation, 1, &col.r);
172     glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
173     glDrawArrays(GL_PATCHES, 0, 16);
174     glDisableVertexAttribArray(0);
175
176     glFinish();
177 }
178
179 void InitVB0s( )
180 {
181     Vec3f Vertices[16] = { // 꼭면의 제어점 좌표
182         {1.5, -1.5, 0.0}, {1.5, -0.5, 0.2},
183         {1.5, 0.5, 0.0}, {1.5, 1.5, -1.0},
184
185         {0.5, -1.5, 0.5}, {0.5, -0.5, 2.0},
186         {0.5, 0.5, 2.0}, {0.5, 1.5, 1.0},
187
188         {-0.5, -1.5, 0.5}, {-0.5, -2.0},
189         {-0.5, 0.5, 2.0}, {-0.5, 1.5, 1.0},
190
191         {-1.5, -1.5, 0.0}, {-1.5, -1.0},
192         {-1.5, 0.5, 0.5}, {-1.5, 1.5, -1.0} };
193
194     glGenBuffers(N_VB0s, VB0);
195     glBindBuffer(GL_ARRAY_BUFFER, VB0[B_SURFACE]);
196     glBufferData(GL_ARRAY_BUFFER, sizeof(Vertices),
197                 Vertices, GL_STATIC_DRAW);
197     glPatchParameteri(GL_PATCH_VERTICES, 16);
198
199     glEnable(GL_DEPTH_TEST);
200     glEnable(GL_POLYGON_OFFSET_FILL);

```

```

201     glPolygonOffset(1.0, 1.0);
202 }
203
204 int main(int argc, char** argv)
205 {
206     glutInit(&argc, argv);
207     glutInitDisplayMode(GLUT_RGBA | GLUT_DEPTH);
208     glutInitWindowPosition(50, 100);
209     glutInitWindowSize(500, 500);
210     glutCreateWindow("베지어 스플라인 곡면");
211
212     GLError s = glewInit();
213     if (s != GLEW_OK) {
214         fprintf(stderr, "오류 - %s\n", glewGetErrorString(s));
215         return 1;
216     }
217
218     SetUpShaders( );
219     InitVB0s( );
220
221     glutDisplayFunc(RenderCB);
222     glutMainLoop( );
223
224     return 0;
225 }

```

(1) 테셀레이션 제어 셰이더

먼저 테셀레이션 제어 셰이더는 테셀레이션 패치(patch)의 꼭지점들을 입력받아 얼마나 많은 기하요소가 발생되어야 하는지 지정한다. 하나의 패치가 몇 개의 꼭지점으로 구성되는가는 `glPatchParameteri` 함수로 알린다.

```
void glPatchParameteri(GLenum pname, GLint value);
```

여기서 pname에는 GL_PATCH_VERTICES를 사용하고 value에는 꼭짓점의 수를 전달한다. 그리기 함수(glDrawArray 등) 1회 호출에 의해 처리되는 패치들은 모두 같은 개수의 꼭짓점을 갖는다.

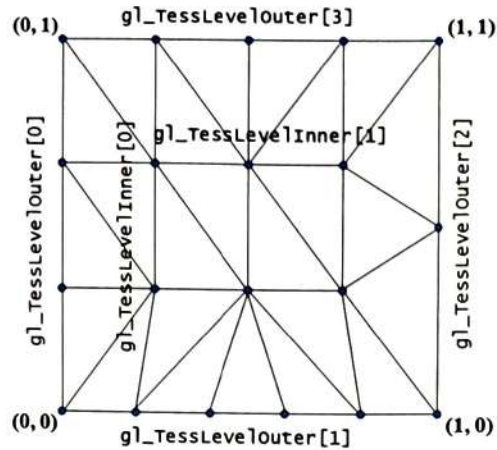
테셀레이션 제어 셰이더는 테셀레이션 계산 셰이더에 전달할 테셀레이션 출력 패치 꼭짓점들을 발생시킨다. 이때, 필요하다면 꼭짓점 속성이나 패치 속성값에 적절한 처리를 할 수 있다. 또한 기본 요소 발생기의 동작을 제어하기 위한 테셀레이션 레벨 인자(tessellation level factors)를 지정한다. 3차 베지어 스플라인 곡면을 생성하려면 4×4개의 제어점으로 구성되는 사각형 패치를 사용하는데, 이 경우 gl_TessLevelOuter[0]~gl_TessLevelOuter[3]의 4개의 인자와 gl_TessLevelInner[0]와 gl_TessLevelInner[1]이라는 2개의 인자를 지정한다. 이 인자들은 단위 정사각형을 작은 삼각형으로 채워 넣는데, 이때 몇 개의 구획으로 분할할지를 나타낸다. gl_TessLevelOuter는 바깥 둘레의 구획 수를 나타내며, gl_TessLevelInner는 각각 수직 및 수평 방향으로 몇 개의 내부 영역이 존재하는가를 나타낸다. [그림 8-34]는 다음의 인자에 의해 어떻게 테셀레이션이 이루어지는가를 보여 준다.

```
gl_TessLevelOuter[0] = 3;
gl_TessLevelOuter[1] = 5;
gl_TessLevelOuter[2] = 2;
gl_TessLevelOuter[3] = 4;
gl_TessLevelInner[0] = 3;
gl_TessLevelInner[1] = 4;
```

[소스코드 8-2]의 테셀레이션 제어 셰이더 소스에서 27행의 'vertices = 16'은 테셀레이션 제어 셰이더가 출력하는 패치의 꼭짓점 수가 16개라는 것을 나타낸다. 즉, 꼭짓점 셰이더에서 제공되는 꼭짓점 16개를 모아 하나의 패치를 구성하는데, 이 16개의 꼭짓점으로 4×4개의 제어점을 전달할 것이다.

31~36행에서 테셀레이션 레벨 인자를 모두 16으로 지정함으로써 가로와 세로로 균일하게 16개씩 나누도록 한다.

38~39행에서 gl_in은 입력되는 패치의 꼭짓점들의 속성을 저장하는 배열이고, gl_out은 출력되는 패치의 꼭짓점 배열이다. gl_InvocationID는 그 패치의 몇 번째 꼭짓점을 셰이더가 처리하고 있는지를 나타내는 변수이다.



[그림 8-34] 테셀레이션 레벨 인자

(2) 테셀레이션 계산 셰이더

테셀레이션 계산 셰이더는 테셀레이션 제어 셰이더로부터 테셀레이션 기본 요소 발생기를 통해 전달된 각 좌표에 대해 동작하여 이 좌표에 대한 꼭짓점 위치를 결정한다. [소스코드 8-2]의 테셀레이션 계산 셰이더에서 44행의 `layout` 문장은 사각형을 삼각형으로 분할해야 하고(quad), 모서리는 동일 크기로 균일하게 분할해야 하며(equal_spacing), 시계 반대방향 순서로 삼각형을 생성함을 나타낸다.

47~51행의 함수 `B`는 베지어 스플라인의 배합함수인 Bernstein 다항식을 구하는 함수로, 62행의 `B(j, u)`는 식 (8-23)의 $BEZ_{3,j}(u)$, `B(k, v)`는 $BEZ_{3,k}(v)$ 에 해당된다. `gl_TessCoord`라는 내장변수를 통해 제공되는 지점에 해당되는 좌표를 60~64행의 명령으로 계산한 후, 유니폼 변수 `gTransform`을 통해 전달받은 변환행렬을 곱하여 뷰잉 및 투영 결과 좌표를 구한다.

이와 같이 구성된 셰이더 소스 프로그램들은 `SetUpShaders` 함수를 이용해 빌드하여 GPU에 전달한다. 135, 136행에서는 뷰잉 및 투영을 위한 변환행렬에 해당되는 유니폼 변수의 위치와 프래그먼트 셰이더에 전달할 색을 저장하기 위한 유니폼 변수의 위치를 조회한다.

(3) 제어점의 전달 및 곡면 생성

179행의 `InitVB0s` 함수에서는 꼭짓점 버퍼 객체를 생성하여 16개의 꼭짓점 좌표를 전달한다. 이때, 197행에서 `glPatchParameteri` 함수로 패치의 꼭짓점 수(즉, 제어점 수)를 16개로 지정하였다.

렌더링 콜백 함수 `RenderCB`에서는 159, 160행에서 뷰잉 변환행렬(`viewMat`)과 직교투영행렬(`orthoMat`)을 곱한 변환행렬을 유니폼 변수에 전달(161행)한 다음 169행과 173행의 `glDrawArrays` 함수로 렌더링을 지시한다. 이때, 'GL_PATCHES'를 지정하여 함수를 호출한다.

[그림 8-35]는 이 프로그램을 실행한 결과이다.



[그림 8-35] OpenGL의 테셀레이션 셰이더를 이용하여 생성한 3차 베지어 곡면

요약

- 3차원 객체는 경계표현, 공간분할표현, 구조적 입체기하, 스위핑 등의 방식으로 모델링할 수 있다.
- 스플라인 곡선은 제어점에 의해 결정되는 매개변수 다항식으로 정의한다.
- 스플라인 곡선이 얼마나 부드럽게 연결되는가를 나타내는 속성을 연속성이라고 한다. n 차 매개변수 연속성이 만족되면 n 차 기하학적 연속성도 만족한다.
- 에르미트 곡선이나 베지어 곡선은 G^0 , G^1 , C^1 을 만족할 수 있도록 곡선을 연결할 수 있다.
- 에르미트 스플라인 곡선은 곡선의 양 끝점에 해당하는 2개의 제어점과 이 두 점에서의 1차 미분에 의해 정의된다.
- n 차 베지어 곡선은 n 개의 제어점으로 정의하며, 처음과 마지막 제어점을 연결하고 나머지 제어점을 근사화하는 스플라인 곡선이다.
- 베지어 곡선은 제어점에 의해 형성되는 볼록 껍질 내부에 포함되며, 아핀 불변성을 갖는다.
- $n+1$ 개의 제어점에 의해 정의되는 d 차 B-스플라인 곡선은 $d+n+1$ 개의 매개변수 구간($d+n+1$ 개의 매듭)으로 설계된다.
- d 차 B-스플라인은 C^{d-1} 연속성을 갖는 곡선을 생성한다.
- 균일 B-스플라인은 매듭의 간격이 동일하며, 제어점을 근사화하는 스플라인 곡선을 정의한다.
- 비균일 B-스플라인은 매듭의 간격이 일정하지 않으며, 조밀하게 배치된 매듭 방향으로 곡선을 끌어당기는 효과가 있다.
- 개방 B-스플라인은 처음과 마지막 제어점을 연결한다.
- NURBS는 유리함수 형태의 배합함수를 사용하며, 가중치를 포함하는 제어점 리스트, 비균일 매듭 벡터로 정의된다.
- NURBS는 아핀 불변성뿐 아니라 원근투영 불변성도 성립하며, 원뿔곡선을 그릴 수 있다.

연습문제

객관식

- ① 8진 트리 ② CSG
③ NURBS ④ 스위핑

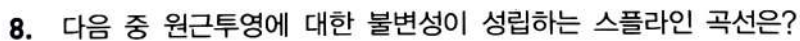
- $$(\sqrt{x^2 + y^2} - r_z)^2 + z^2 = r^2$$



- ① G^0 연속성은 두 곡선의 연결점에서 1차 미분이 비례한다.
- ② G^1 연속성을 만족할 경우 C^1 연속성도 만족된다.
- ③ G^2 연속성을 만족할 경우 G^1 연속성도 만족된다.
- ④ G^0 연속성은 두 곡선이 서로 연결되지 않음을 의미한다.

- ① 2개의 끝점과 두 끝점에서의 1차 미분
- ② 4개의 제어점
- ③ 2개의 끝점과 1개의 중간점
- ④ 4개의 끝점과 2개의 매듭값

7. 개방 3차 B-스플라인 곡선에 해당되는 그림은?



- 정답 1. ② 2. ④ 3. ③ 4. ① 5. ② 6. ③ 7. ① 8. ②

주관식

1. 두 끝점이 $p_1 = (1, 2)$, $p_2 = (10, 3)$ 이고, p_1 에서의 1차 미분이 $(3, 9)$, p_2 에서의 1차 미분이 $(9, -6)$ 인 에르미트 곡선을 10개의 매개변수 구간으로 나누어 좌표를 계산하고, 이를 선분으로 이어 곡선을 그려 보라.
2. 문제 1과 동일한 곡선을 정의하는 3차 베지어 곡선의 제어점은 무엇인가?
3. n 차 베지어 곡선의 양 끝점에서의 1차 미분을 구하라.
4. 균일 2차 B-스플라인 곡선은 4개의 제어점과 7개의 매듭값으로 정의된다. 매듭 벡터가 $\{0, 1, 2, 3, 4, 5, 6, 7\}$ 일 때, 균일 2차 B-스플라인 곡선의 배합함수를 구하라.